幾何音響シミュレーション 技術説明書

— プログラムのフローと数式の解説 —

最終更新: 2026-08-23（HBN240018）

0. この資料について

`geosim/` が何をしているかを、**プログラムの流れ**と**式の意味**の両面から解説する資料です。「この式は法線を出している」「この式は法線と音線から交点を求めている」というレベルまで噛み砕いて書いています。

- 対象コード: `geosim/`（Fortran 3 ファイルからの移植版）
- 理論的な下敷き: **豊田政弘『建築音響物理学』関西大学出版部 2.2 節（幾何音響理論）**

  移植元 Fortran コードは、事実上この本の 2.2 節の実装です。以下、式番号は断りがなければこの本のもの。
  抜粋 PDF の場所は `参考文献/README.md` を参照（PDF は Git 管理外）。
- 実装状況は `PROGRAM_STRUCTURE.md`、残作業は `TODO.md` を参照。
- 高速化の経緯と実測値は `docs/高速化の説明.md`（この資料は**式の意味**に集中する）。
- **PDF 版が `docs/pdf/` にある**（作り直しは `python docs\build_pdf.py`）。

2026-08-12 以降に入った大きな変更（この資料に反映済み）

いつ	何が変わったか	この資料のどこ
10-30	**最大反射回数を応答の長さから決める**（試し追跡。固定の 120 回だと小さい室で足りなかった）	4.3 節
10-21	**経路の照合を平面で行う**（同じ平面の別レイヤに当たる経路を捨てない）	7.2 節
10-21	**音線の本数を応答の長さから決める**（指定もでき、足りなければ「精度が怪しい」）	4.3 節
08-19	**交差判定を「同一平面パッチ」単位にした**（三角形単位から）	5.4.1 節
08-19	**容積を法線から出す**（発散定理）。巻き順に依存しない	3.4 節
08-17	統計残響式から**ミリントンの式を落とした**（実務で使わない）	9.6 節
08-21	**音圧レベル**（絶対値）と **STI** を出せるようにした	9.8 / 9.9 節
08-21	**吸音材は「条件表」（xlsx）で決める**。CAD を触らずに差し替えられる	9.5 節
08-21	**経路の幾何を保存して使い回す**（吸音だけ変えた再計算）	9.10 節
08-21	インパルス応答の合成を**時間領域に置いてから FFT** に（65 倍）	8.2.1 節
08-23	査読論文（式(2)）との一致を位相まで確認し、**厳密版も選べる**ようにした	8.2.1 / 11 節
08-23	**測定点（音源・受音点）の一覧と配置図**を出すようにした	`docs/出力・可視化方針.md`

手法の位置づけ

本 PJ は **音線法（ray tracing）と虚像法（image source method）を組み合わせた 2 段構え**です。

	役割	長所	短所
音線法	「どの壁をどの順に反射する経路が受音点に届くか」を**探す**	任意形状で使える	受音球の大きさ次第で経路を取りこぼす／偽検出する（図2.16, 2.17）
虚像法	見つかった経路について、到来時刻とエネルギーを**正確に計算する**	幾何学的に厳密	虚音源が指数的に増え、総当たりは非現実的（式2.66）

音線法で経路の「当たり」をつけ、虚像法で厳密に計算し直す。これによって 「音線法の精度の甘さ」と「虚像法の計算量爆発」の両方を回避しています。 音線法の段階では**エネルギーを計算していない**のがポイントで、経路探索に徹しています。

---

1. 全体フロー

図（mermaid の記述。GitHub 上では図として表示されます）

```
flowchart TD
    IN["入力<br/>音源位置・受音点・DXF形状<br/>受音球半径・反射回数・音線数・吸音率"]

    S1["① read_dxffile.read()<br/>DXF → 三角形メッシュ<br/><b>法線ベクトルを外積で算出</b>"]
    S2["② sound_ray.soundray_generator()<br/>音源から全方位へ音線を生成<br/><b>球面上に等立体角で分布</b>"]
    S3["③ loop_reflectionmesh.loop()<br/><b>音線追跡</b><br/>交差判定→受音判定→鏡面反射<br/>受音した経路の反射面ID列を記録"]
    S4["④ loop_deleteredundancy.delete()<br/>重複経路の削除<br/>同じ壁面列は1本にまとめる"]
    S5["⑤ loop_noredundancy.loop()<br/><b>虚音源バックトレース</b><br/>到来時刻とバンド別エネルギーを算出"]
    S6["⑥ impulse.impulse_response_from_pulses()<br/>パルス列 → インパルス応答<br/>空気吸収・バンド合成"]
    S7["⑦ reverberation.reverberation_time()<br/>Schroeder 積分 → 減衰曲線<br/>残響時間 T30"]

    IN --> S1 --> S2 --> S3 --> S4 --> S5 --> S6 --> S7

    S3 -. "経路を探す<br/>（音線法）" .- N1[ ]
    S5 -. "厳密に計算する<br/>（虚像法）" .- N2[ ]

    style S3 fill:#e8f0fe,stroke:#4a6fa5
    style S5 fill:#e8f0fe,stroke:#4a6fa5
    style N1 fill:none,stroke:none
    style N2 fill:none,stroke:none
```

書籍の図2.15（音線法のフローチャート）が ①〜③、図2.20（虚像法のフローチャート）が ⑤〜⑥ に対応します。

---

2. 記号の約束

以降で使う記号です。コード上の変数名・Fortran の変数名も併記します。

記号	意味	Python	Fortran
$\mathbf{p}_0$	音線の**基点**（未反射なら音源位置、反射後は直前の反射点）	`soundray_comesfrom`	`vini`
$\mathbf{v}$	音線の**進行方向単位ベクトル**	`sound_ray`	`vray`
$\mathbf{n}$	壁面の**内向き単位法線ベクトル**	`mesh[j].normal`	`vwnm(j,:)`
$\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3$	三角形メッシュの頂点座標	`mesh[j].vertexes`	`vwpt(surf(j,2..4),:)`
t	交点パラメータ（$\mathbf{p}_0$ から交点までの距離）	`t`	`temp`
$\mathbf{q}$	音線と壁面の**交点**	`node`	`node`
$\mathbf{x}_r$	受音点	`receiver_point`	`(rcvx, rcvy, rcvz)`
r_c	受音球の半径	`sphere_radius`	`rcvr`
α_n	**垂直入射**吸音率（バンド別）	`absorption_coefficient`	`absper(id, 1..6)`
θ	入射角（法線となす角）	—	—
E	音線のエネルギー（バンド別）	`soundray_energy`	`enertmp(6)`
c	音速 [m/s]	`sound_velocity`	`c0`

ベクトルはすべて 3 次元の列ベクトル、$\mathbf{a}\cdot\mathbf{b}$ は内積、$\mathbf{a}\times\mathbf{b}$ は外積です。

> **なぜ法線を「内向き」に揃えるのか**
> 室内音響では音は閉じた空間の内側を飛びます。法線を室内側に揃えておくと、
> 「音線が壁に向かっている」という判定が $\mathbf{v}\cdot\mathbf{n} < 0$ という**符号ひとつ**で書けます（3.1 節）。
> 向きがバラバラだと面ごとに場合分けが必要になります。

---

3. ① 室形状の読み込みと法線ベクトル

対応コード: `read_dxffile.py` ／ `mesh.py`　｜　書籍: p.79 図2.12 CAD 側の作図ルールは `docs/DXFデータの作り方.md` を参照

3.1 三角形メッシュ

室形状を三角形の集まりで表します。三角形に限定するのは、「平面であることが常に保証される」「頂点 3 点で法線が一意に決まる」ためです。四角形以上だと 4 点が同一平面上にない場合があり、交差判定が破綻します。

3.2 法線ベクトルを外積で求める ← 「この式は法線を出している」

頂点 $\mathbf{x}_1$ を基準に、三角形の 2 辺を表すベクトルを作ります。

$$\mathbf{v}_{12} = \mathbf{x}_2 - \mathbf{x}_1, \qquad \mathbf{v}_{13} = \mathbf{x}_3 - \mathbf{x}_1$$

この 2 本の**外積**が法線です。

$$\mathbf{n}' = \mathbf{v}_{12} \times \mathbf{v}_{13} = \begin{pmatrix} v_{12,y}v_{13,z} - v_{12,z}v_{13,y} \\ v_{12,z}v_{13,x} - v_{12,x}v_{13,z} \\ v_{12,x}v_{13,y} - v_{12,y}v_{13,x} \end{pmatrix}$$

外積の意味：外積は「2 本のベクトルが張る平面に垂直で、右ねじの向き」のベクトルを返します。$\mathbf{v}_{12}$ も $\mathbf{v}_{13}$ も三角形の面内にあるので、それに垂直＝**面の法線**になります。

長さを 1 に正規化します。

$$\mathbf{n} = \frac{\mathbf{n}'}{\|\mathbf{n}'\|}$$

正規化する理由：以降の式で $\mathbf{n}$ を単位ベクトルとして扱うためです。たとえば $\mathbf{n}\cdot\mathbf{a}$ が「$\mathbf{a}$ の法線方向成分の長さ」そのものになり、式が簡単になります。

押さえておく点が 3 つあります。

1. **向きは頂点の順序で決まる。** $\mathbf{x}_1 \to \mathbf{x}_2 \to \mathbf{x}_3$ が反時計回りに見える側が $\mathbf{n}$ の向きです。

   CAD 側の頂点順序が揃っていないと法線が外向きになる面が混ざるので、閉空間なら内向きに揃える必要があります。
2. $\|\mathbf{n}'\| = 0$ **なら縮退面。** 3 点が一直線上に並んでいる（＝面積ゼロの三角形）ケースで、

   正規化でゼロ除算になります。元 Fortran コードはここで面を捨てています。
3. **外積の大きさは平行四辺形の面積。** $\|\mathbf{n}'\|/2$ が三角形の面積になるので、

   吸音力の計算などで面積が要るときはここで取れます。

3.3 法線の向き — モデル側で正しく作る

面は片側だけ反射します。 音線が壁に当たったと判定されるのは $\mathbf{v}\cdot\mathbf{n} < 0$（法線に向かって進んでいる）のときだけなので、**法線の裏側から来た音線はその面を通り抜けます**（3.1 節）。

したがって作図のルールはこう決まります。

> **反射させたい空気の側に法線を向ける。**

対象	法線の向き	空気はどこにあるか
室の外殻	室内向き	内側
室内に置く物体（机・什器）	外向き	物体の外側
一面だけの反射板	反射させたい側	その側

この向きの正解を持っているのは CAD モデル自身なので、読み込み側は既定で CAD の巻き順をそのまま使います（`orient_normals='cad'`）。勝手な補正はしません。

> ⚠ **「法線を音源方向に向ける」のは誤りです。** 凸凹のある壁や宙に浮いた家具では
> 向きが破綻して衝突判定がおかしくなります。

両面で反射させたいものは厚みを持たせる

面は片側だけ反射するので、**1 枚の面で両面反射はできません。** 両側から音が当たる物体は、CAD 側で**厚みのある立体**として作ります。

厚みのある机（脚は省略）なら、**上面は上向き法線・下面は下向き法線**となり、どちらも空気側を向きます。机の上を通る音線は上面で反射し、下を通る音線は下面で反射します。

向きの誤りを見つける ― シェル診断

読み込み時に、辺の共有で**シェル（連結した面のかたまり）**に分割し、 シェルごとに閉/開・体積・法線の向きを報告します。

閉じたシェルの法線の向きは**符号付き体積**で判定できます。

$$V = \frac{1}{6}\sum_{\text{¤}} \mathbf{x}_1 \cdot (\mathbf{x}_2 \times \mathbf{x}_3)$$

各三角形と原点が作る四面体の符号付き体積の和です。巻き順が一貫した閉曲面なら、 $V > 0$ で法線が外向き、$V < 0$ で内向きと決まります。**重心との内積で面ごとに判定する方法と違い、 凸形状に限定されない**のが利点です（重心方式は L 字型の室のように重心が壁の裏に来る形状で誤判定します）。

そして望ましい向きはシェルの位置で決まります。

- **外殻**（他のすべてを内包するシェル）… `inward`（$V < 0$）が空気側
- **内側のシェル**（机など）… `outward`（$V > 0$）が空気側

室 4×5×3 m ＋ 厚みのある机 1.6×0.8×0.05 m の診断出力：

```
シェル0 [外殻] 面12枚 閉 体積60.0000m³ 法線=inward  OK
シェル1 [内側] 面12枚 閉 体積 0.0640m³ 法線=outward OK
```

巻き順が一貫していない（隣り合う面で法線が反対を向いている）場合も検出できます。 **同じ有向辺が 2 回現れるか**を見ればよく、巻き順が揃った面同士が辺を共有するとき その辺は互いに逆向きに現れるはず、という性質を使います。

なお、法線ベクトルだけを反転すれば済み、頂点の並び順を直す必要はありません。 5.4 節の内外判定は外積の内積の符号を見ているだけで、巻き順を反転しても値が変わらないためです。

閉じた室でなくてもよい

面の片側性のおかげで、**閉じた形状である必要はありません。** 一面だけの反射板を置いて「直接音と 1 回反射音だけが到来する状況」を検証できます （音線は反射後に飛び去り、当たる壁がなくなった時点で追跡が打ち切られます）。

> 元コード（`backtrace.f90` 276〜282 行）は `ynnmrev` という y/n フラグで全法線を反転するかを
> ユーザーに聞くだけでした。シェル診断と `'shells'` モードは Python 版で追加した機能です。
> 詳細は `docs/DXFデータの作り方.md` 3 節。

3.4 容積を法線から出す（発散定理）

統計残響式（9.6 節）には室の**容積**が要ります。閉じた形状なら形から出せます。

$$V = \frac{1}{3}\oint_S \mathbf{r}\cdot\mathbf{n}\,dA \approx \frac{1}{3}\sum_j (\mathbf{c}_j \cdot \mathbf{n}_j)\,A_j$$

$\mathbf{c}_j$ は三角形 j の重心、$\mathbf{n}_j$ は**内向き**の単位法線、 A_j は面積です（内向きなので符号を反転して使う）。発散定理 $\int_V \nabla\cdot\mathbf{r}\,dV = \oint_S \mathbf{r}\cdot\mathbf{n}\,dA$ と $\nabla\cdot\mathbf{r} = 3$ から出ます。

★巻き順から出す符号付き体積 `V=\frac16\sum \mathbf{x}_1\cdot(\mathbf{x}_2\times\mathbf{x}_3)`（3.3 節）**ではなく法線から出す** のが要点です（2026-08-19 に変えた）。符号付き体積は**巻き順が一貫した閉曲面でしか 成り立たない**ので、リージョン変換したモデルや、壁を帯・開口で分割したモデル （T 字接合で「開いた辺」が出る）では合いません。法線なら T 字接合でも正しく、 **家具・反射板の体積も自動で引かれます**（内側の閉じた物体は法線が外を向くため）。

ただし**穴があると誤る**ので、`encloses_point()` で「本当に囲んでいるか」を 確かめてから採用します。符号付き体積は検算用に残してあり、1% 以上食い違えば `volume_note` で知らせます。

対応コード: `read_dxffile.volume_from_normals()` / `signed_volume()`（検算用）

> **「開いた辺」は 2 種類あるので混ぜない。**同じ直線上の他の開いた辺に覆われて
> いれば **T 字接合**（面は閉じている。壁を分割しただけ）、覆われていなければ
> **本当の自由端**（宙に浮いた片面の板・面の抜け）です。自由端があれば発散定理は
> 成り立たないので「目安」と断ります（ `uncovered_open_edges()` ）。

---

4. ② 音源からの音線生成

対応コード: `sound_ray.soundray_generator()`（元コード 318〜326 行） 書籍: p.78 図2.11、補遺 A.15（立体角）

4.1 何をしたいか

無指向性音源を表現するには、音源から**全方位に均等に**音線を飛ばす必要があります（書籍 図2.11）。 ここでいう「均等」とは**等立体角**、つまり球面上で音線の到達点が一様分布になることです。

緯度・経度を単純に等分割すると極付近に音線が密集してしまうので、それではいけません。

4.2 Fibonacci 螺旋（黄金角法）

コードでは次の式で N 本の単位ベクトルを作っています（$i = 0, 1, \ldots, N-1$）。

$$z_i = \frac{2i}{N} - 1 + \frac{1}{N}$$

$$\varphi_i = i \cdot \Delta, \qquad \Delta = \pi(3 - \sqrt{5}) \approx 2.3999 \text{ rad } (\approx 137.5^\circ)$$

```
\mathbf{v}_i = \begin{pmatrix} \sqrt{1 - z_i^2}\,\cos\varphi_i \\ \sqrt{1 - z_i^2}\,\sin\varphi_i \\ z_i \end{pmatrix}
```

なぜこれで等立体角になるのか（2 つの仕掛け）

- z **を等間隔に刻んでいる**のがポイントです。半径 1 の球を高さ z で薄く輪切りにしたとき、

  その帯の面積は $2\pi\Delta z$ で、z **の位置によらず一定**です（アルキメデスの球帯の定理）。
  つまり z を等間隔にとれば、それだけで立体角が等分されます。極に密集しません。
- Δ **が黄金角**（`\pi(3-\sqrt5)`）であることで、経度方向がどこまで回しても既出の角度と重ならず、

  螺旋状に隙間なく埋まります。黄金比が無理数であることを使った、ヒマワリの種と同じ配置です。

$\sqrt{1 - z_i^2}$ は、高さ z_i で球を切ったときの**断面円の半径**です。 その円周上を角度 φ_i の方向に取る、というのが x, y 成分の意味です。

z_i の式の $+1/N$ は、z が -1 や $+1$（真南・真北の極）にぴったり来ないよう半目盛ずらすためのものです。 極では断面円が点に潰れてしまうので、それを避けています。

> **Fortran 版のバグ**：元コードは $i = 1, \ldots, N$（1 始まり）でこの式を使っているため、
> $i = N$ のとき $z = 1 + 1/N > 1$ となり $\sqrt{1 - z^2}$ が NaN になります。
> Python 版は 0 始まりなので $z \in [-1 + 1/N,\ 1 - 1/N]$ に正しく収まります（2026-08-12 検証済み）。

> **代替手法**： `参考文献/02_音線生成・方向離散化/` に「軸対称等立体角26面体を用いた全方位の離散化」があります。
> 音線数が少ないときは Fibonacci 螺旋より均等性が良い可能性があり、比較検討の余地があります。

4.3 ★音線の本数は応答の長さから決める（2026-10-21）

元の作成者 豊田政弘氏の修正版 `backtrace_rev.f90`（2026-10-19）の改善 3 に合わせました。

虚音源は、そこへ向かう音線が受音球を **1 本でも**通ったときに初めて見つかります。 音線は音源から全方向に広がるので、遠く（遅い時刻）ほど間隔が開きます。 距離 ct の虚音源から半径 r の受音球に届く本数の期待値は（Vorländer 1989 の式(8)）

$$k(t) = \frac{N r^2}{4(ct)^2}$$

1 本も届かない確率は e^{-k} です。そこで「応答の長さ $t_{\max}$ の時刻で k 本は 届いてほしい」から本数を決めます（同 式(9)）。

$$N = \frac{4k\,(c\,t_{\max})^2}{r^2}$$

設定	本数	期待値 1 本の時刻
修正版の既定（3 s・3 本・r = 1 m・c = 340）	12,484,800 本	5.2 s
以前の既定 20 万本・r = 1 m	200,000 本	0.65 s（3 s では 0.05 本＝95% 見落とす）
JR の案件 100 万本・r = 1 m	1,000,000 本	1.46 s（5 s では 0.09 本）

- 設定は `rays_mode`（`auto` 自動 ／ `manual` 指定）と `rays_hits`（自動のときの k。既定 3）。

  ★計算時間を優先して本数を指定することもできます（2026-10-21 ユーザー指示）。
  そのときは応答の長さの時刻で期待値が **1 本を切ると「精度が怪しい」**と、

設定画面（赤字）・実行ログ・`計算情報.csv` に出します。期待値 1 本の時刻より後の
反射音は多くを見落とすので、後半の残響（T20 / T30）は当てにしないでください

- **★昔の project.json（`rays_mode` が無い）と、本数を渡して作ったプロジェクトは「指定」**で読みます。

自動で読むと、100 万本で回していた案件が黙って数千万本になるためです

- **★伝搬距離が $ct_{\max}$ を超えた音線は追跡をやめます**（`loop_reflectionmesh.loop(max_length=…)`）。

応答の外に届く経路は使わないので、その分の計算を省くだけです。受音もこの距離まで
（経路の長さ − 受音半径 ≦ $ct_{\max}$）に限ります。最大反射回数は**安全装置**になり、
応答の長さに届く前に上限で止まった音線の数を実行ログに出します（多ければ増やす）

対応コード: `ray_count.py`（`rays_for` / `expected_hits` / `plan`）／ `run_project._freeze_ray_count()`（実行の頭で本数を確定）

★最大反射回数も応答の長さから決める（2026-10-30）

豊田氏の修正版の 10-23 版（改善 7）に合わせました。伝搬距離 $ct_{\max}$ で打ち切っても、 それより先に**最大反射回数で止まってしまえば応答の後半は欠けます**。平均自由行程 $\bar{\ell} = 4V/S$ の室では、時刻 t までの反射回数はおよそ $ct/\bar{\ell}$ 回なので、固定の回数では小さい室ほど足りません。

室	平均自由行程	120 回で届く時刻の目安	5 s までの反射回数（試し追跡）
JR 研修室（556 m³）	3.8 m	約 1.5 s	平均 449 回・最大 777 回

実際、研修室を 500 万本で回すと**ほぼ全部の音線が 120 回の上限で止まり**、 T20 が 3〜4 s の条件（壁が硬い条件）は後半が欠けていました。

そこで、方向を均等に散らした 10 万本を**伝搬距離が $ct_{\max}$ に達するまで反射回数の制限なしで** 追い（試し追跡。受音は見ない）、その最大反射回数に余裕（既定 1.25）を掛けた値を上限にします。 研修室なら $\lceil 1.25 \times 777 \rceil = 972$ 回です。

- 設定は `nref_mode`（`auto` 自動 ／ `manual` 指定）と `nref_margin`（余裕。既定 1.25。

豊田氏のメールによると「AI の推奨値」。豊田氏自身は正確さ重視で 10）

- **★昔の project.json（`nref_mode` が無い）と、最大反射回数を渡して作ったプロジェクトは「指定」**で読みます

（音線数と同じ約束。結果が黙って変わらないように）

- 試し追跡は幾何・音源・応答の長さが同じなら使い回します（条件をまたいで 1 回。研修室で約 4 分）
- 室外へ抜けた音線（形状のすき間）と、100 万回に達した音線（閉じ込め）は数えて知らせます

対応コード: `loop_reflectionmesh.sample_reflections`（試し追跡）／`ray_count.reflection_limit`／`run_project._freeze_reflection_limit()`（実行の頭で上限を確定）

---

5. ③ 音線追跡

対応コード: `loop_reflectionmesh.py`（元コード 524〜717 行）／ `mesh_method.py` ／ `receiver_sphere.py` 書籍: p.79〜85

この節が本 PJ の心臓部です。ループ構造は 3 重になっています。

```
音線ループ i = 1 … N本            ← 音源から出る音線を1本ずつ
  反射回数ループ k = 0 … nref      ← その音線が反射を繰り返す
    壁面ループ j = 1 … 面数        ← どの壁に当たるかを全面で調べる
      5.1 向き判定 → 5.2 平面の式 → 5.3 交点t → 5.4 内部判定 → 5.5 最寄り面
    5.6 受音判定
    5.7 鏡面反射して次へ（当たる壁がなければ打ち切り）
```

5.1 音線が壁に向かっているか ← 背面カリング

$$\mathbf{v} \cdot \mathbf{n} < 0$$

意味：内積は $\mathbf{v} \cdot \mathbf{n} = \|\mathbf{v}\|\|\mathbf{n}\| \cos\phi = \cos\phi$ （どちらも単位ベクトル）。法線は室内向きなので、音線が壁に向かって進んでいるとき $\mathbf{v}$ と $\mathbf{n}$ は逆向き寄りになり、$\cos\phi < 0$ になります。

逆に `\mathbf{v}\cdot\mathbf{n} \ge 0` なら、その壁は音線の**裏側を向いている**か平行なので、 交差の計算をするまでもなく除外できます。全面に対して交点を計算するのは無駄なので、 ここで安く足切りしています（コンピュータグラフィックスでいう背面カリング）。

なお、この後の交点パラメータの式では $\mathbf{v} \cdot \mathbf{n}$ が**分母**に来るので、 この判定はゼロ除算（音線が壁と平行なケース）を防ぐ役割も兼ねています。

5.2 壁面を含む平面の方程式

平面は次の式で表せます。

$$\mathbf{n}\cdot\mathbf{x} + d = 0 \qquad \text{¤¤¤} \qquad n_x x + n_y y + n_z z + d = 0$$

定数 d は、平面上の点（頂点なら何番でもよい）を代入して決めます。

$$d = -\,\mathbf{n}\cdot\mathbf{x}_1$$

意味：点 $\mathbf{x}$ が平面上にある条件は、「基準点 $\mathbf{x}_1$ からのベクトル $\mathbf{x}-\mathbf{x}_1$ が 法線と直交すること」です。式にすると

$$\mathbf{n}\cdot(\mathbf{x}-\mathbf{x}_1) = 0 \quad\Longleftrightarrow\quad \mathbf{n}\cdot\mathbf{x} - \mathbf{n}\cdot\mathbf{x}_1 = 0$$

これがまさに $d = -\mathbf{n}\cdot\mathbf{x}_1$ とおいた形です。頂点はどれを使ってもよいのは、 3 頂点すべてが同じ平面上にあるからです。

$\mathbf{n}$ が単位ベクトルのとき、 $\mathbf{n}\cdot\mathbf{x} + d$ は**点 $\mathbf{x}$ から平面までの符号付き距離**になります。 これが次の式で効いてきます。

対応コード: `mesh_method.parameter_d()`

5.3 音線と平面の交点を求める ← 「法線と音線から交点を求めている」式

音線を直線としてパラメータ表示します（t は基点からの距離）。

$$\mathbf{x}(t) = \mathbf{p}_0 + t\,\mathbf{v}$$

これを平面の方程式に代入します。

$$\mathbf{n}\cdot(\mathbf{p}_0 + t\,\mathbf{v}) + d = 0$$

t について解くと、

```
\boxed{\ t = -\,\frac{\mathbf{n}\cdot\mathbf{p}_0 + d}{\mathbf{n}\cdot\mathbf{v}}\ }
```

交点座標は t を戻して、

$$\mathbf{q} = \mathbf{p}_0 + t\,\mathbf{v}$$

式の読み方：

- **分子** $-(\mathbf{n}\cdot\mathbf{p}_0 + d)$ … 基点 $\mathbf{p}_0$ から平面までの符号付き距離（の符号反転）。

  「あとどれだけ離れているか」
- **分母** $\mathbf{n}\cdot\mathbf{v}$ … 音線の進行方向のうち、法線方向の成分。

  「1 進むごとに平面へどれだけ近づくか」
- 割り算して t = **平面に到達するまでの距離**。「距離 ÷ 近づく速さ = 到達までの長さ」という素直な形です。
- $\mathbf{n}$ も $\mathbf{v}$ も単位ベクトルなので、t **はそのまま実距離** [m] になります。

$t > 0$ **の判定**が必要なのは、直線は前後に無限に伸びているためです。 $t < 0$ の交点は音線の**後方**にあり、物理的には当たりません。

対応コード: `mesh_method.parameter_t()` , `mesh_method.node_renew()` 書籍: p.79「その平面と音線との交点 (x_n, y_n, z_n) はパラメータ t を用いて…」

5.4 交点が三角形の内部にあるか（式2.50）

5.3 で求めたのは**無限に広がる平面**との交点なので、それが三角形の内側かを確かめる必要があります。

頂点 $\mathbf{x}_1$ を基準に 3 本のベクトルを作ります。

$$\mathbf{e}_1 = \mathbf{x}_2 - \mathbf{x}_1, \qquad \mathbf{e}_2 = \mathbf{x}_3 - \mathbf{x}_1, \qquad \mathbf{u} = \mathbf{q} - \mathbf{x}_1$$

外積を 2 つ作り、その内積の符号を見ます。

$$I_1 = (\mathbf{u}\times\mathbf{e}_1)\cdot(\mathbf{u}\times\mathbf{e}_2)$$

同じことを頂点 $\mathbf{x}_2$ を基準にもう一度やって I_2 を求め、

```
I_1 \le 0 \quad\text{かつ}\quad I_2 \le 0 \quad\Longrightarrow\quad \text{交点は三角形の内部}
```

なぜこれで内外が判定できるのか

$\mathbf{u}\times\mathbf{e}_1$ と $\mathbf{u}\times\mathbf{e}_2$ は、どちらも三角形の面に垂直なベクトルです（$\mathbf{u},\mathbf{e}_1,\mathbf{e}_2$ はすべて同じ平面内にあるため）。つまり両者は**平行か反平行のどちらか**しかありません。判定はその「どちらか」を内積の符号で見ているだけです。

- $\mathbf{u}$ が $\mathbf{e}_1$ と $\mathbf{e}_2$ が作る角の**内側**にある

  → $\mathbf{u}$ から見て $\mathbf{e}_1$ と $\mathbf{e}_2$ は**逆回り**にある
  → 2 つの外積は逆向き → 内積が**負**
- $\mathbf{u}$ が角の**外側**にある

  → $\mathbf{e}_1$ も $\mathbf{e}_2$ も $\mathbf{u}$ から見て同じ回り
  → 2 つの外積は同じ向き → 内積が**正**

これで「頂点 $\mathbf{x}_1$ の角の内側か」が判定できます。三角形は 3 つの角のすべての内側にある領域ですが、**2 つの角の内側にあれば 3 つ目は自動的に満たされる**（内角の和が $180°$ のため）ので、判定は 2 回で足ります。元コードが頂点 2 と頂点 3 を基準に 2 回だけ計算しているのはこのためです。

`\le 0`（**等号を含む**）なのは、交点が辺や頂点にちょうど乗った場合（外積のどちらかがゼロベクトル→内積 0）を 内側として拾うためです。等号を落とすと、辺に当たった音線が消えてしまいます。

対応コード: `mesh_method.collision_detection()`, `mesh_method.innerproduct_from3vertexes()`

> 書籍 p.80 は、この方法と等価な別解として、$\mathbf{x}_n = a\mathbf{x}_1 + b\mathbf{x}_2 + c\mathbf{x}_3$ の
> 連立方程式を解いて `0\le a,b,c\le 1` を確かめる方法（式2.50、重心座標法）も挙げています。
> 本コードは外積版を採用しています。

5.4.1 ★同一平面パッチ単位で判定する（2026-08-19）

上の 5.4 節は**三角形 1 枚**に対する内外判定です。実際のコードは、**同一平面に並んだ三角形をまとめた「パッチ」**（＝設計者が描いた 1 枚の壁）を 単位にして判定します。

> きっかけはユーザーの指摘：「三角形要素で分割しているのは、あくまでもねじれのない
> 面を定義する必要があるため。ねじれの無い平面であれば、多頂点要素でも良いのでは？」

そのとおりで、**三角形に割るのは「ねじれのない面」を保証するためであって、交差判定の都合ではありません。**判定は 「平面との交点を 1 回求め、外周多角形の内側かを**交差数の偶奇**で数える」形にします。外周は輪に並べ替えず**辺の集まりのまま**扱うので、穴の開いた面もそのまま正しく出ます。

パッチにまとめる条件は 4 つとも満たすこと：

1. **同一平面**（しきい値は 0.1° / 0.1 mm）
2. **辺を共有して連結している**
3. **同じ材料**（まとめると吸音率が引けなくなる）
4. **同じ法線の向き**

★同一平面のしきい値は編集画面（1° / 1 mm）より厳しくする。編集画面の緩さは「壁 1 枚」を選びやすくするための値で、判定に使うと**形が変わります。**

効果は速度（2.3〜3.0 倍）だけではありません。**受音経路が 2.2 倍に増えます**——三角形単位だと「同じ壁の隣の三角形に当たった」だけでバックトレース（7.2 節）が 経路を却下していたためです。

対応コード: `mesh_method.coplanar_patches()` / `PatchArrays` / `collision_arrays()`（`read_dxffile.coplanar_groups()` がグループ分けを行う）

★音線追跡（5 節）とバックトレース（7 節）で必ず同じ入れ物を使うこと。反射面の番号を突き合わせるためです。

5.5 最も近い壁面を選ぶ

音線は複数の壁面と交差しえます（無限に伸ばせば向かい側の壁とも当たる）。実際に当たるのは**いちばん手前**なので、基点からの距離

$$d_j = \|\mathbf{q}_j - \mathbf{p}_0\|$$

が最小になる面 j を選びます。コードでは `min_distance` を ∞ で初期化し、壁面ループで更新していきます。書籍 p.79 の「音線の始点に近いほうの交点を真の交点と判定すればよい」がこれです。

★実際のコードは配列演算（全面を一度に判定する）

上の説明は面を 1 枚ずつ見る形ですが、コードは**全パッチを一度に**判定します（`mesh_method.FaceArrays` / `PatchArrays`）。距離を並べて `argmin` で最も近い面を選ぶので、Python のループは音線 1 本につき反射回数ぶんだけです。

1 枚ずつ見る版（`collision_distance` など）は**参照実装として残してあります。** 数式の意味を追うときはそちらを読み、速度が要る本番は配列版を使う、という分担です。 両者が一致することは `tests/test_geosim.py` の「ベクトル化した交差判定（scalar 版との一致）」で毎回確認しています（**交差判定に手を入れたら必ずこのテストを通すこと**）。

5.6 受音判定 — 受音球（receiving sphere）

対応コード: `receiver_sphere.inside_sphere()`（元コード 649〜663 行）／書籍 p.84、図2.16、図2.17

なぜ「球」なのか：音線は太さのない線、受音点は大きさのない点なので、 線が点を厳密に通過することは数値計算上まず起こりません。そこで受音点のまわりに半径 r_c の球を置き、 **球を貫いた音線を受音とみなします**。この r_c を受音半径と呼びます。

基点から受音点へのベクトルを

$$\mathbf{w} = \mathbf{x}_r - \mathbf{p}_0$$

とすると、判定に使う 2 つの量は次のとおりです。

$$s = \mathbf{v} \cdot \mathbf{w} \qquad \text{□□□□□□□□□□(□)□□□(□)□□□}$$

$$h = \lVert\, \mathbf{p}_0 + s\,\mathbf{v} - \mathbf{x}_r \,\rVert \qquad \text{□□□□□□□□□□□□□□□(□)□□□□□□}$$

- s は、 $\mathbf{w}$ を音線方向へ**正射影**した長さです。 $\mathbf{v}$ が単位ベクトルなので内積がそのまま射影長になります。

  「音線に沿って何 m 進んだところが受音点にいちばん近いか」を表します。
- $\mathbf{p}_0 + s\mathbf{v}$ がその**垂線の足**。そこから受音点までの距離が h で、

  これが受音点と音線（直線）との最短距離です。

受音と判定する条件は、**「基点から最寄り壁までの線分」が球と交わること**です。 そのために、まず垂線の足を線分の中へ押し込みます。

$$s^{*} = \mathrm{clip}(s,\ 0,\ d_{\mathrm{min}})$$

```
\boxed{\ \lVert\,\mathbf{p}_0 + s^{*}\mathbf{v} - \mathbf{x}_r\,\rVert \le r_c\ }
```

s^{*} は「線分の上で受音点にいちばん近い点」の位置です。s が 0 より小さければ 基点そのもの、d_{min} より大きければ壁の当たり点になります。

なぜ垂線の足で判定してはいけないのか ← 2026-09-05 に修正したところ

それ以前は、次の 3 つをすべて満たすことを条件にしていました（元コードのやり方）。

```
\underbrace{h \le r_c}_{\text{①球を貫いた}} \quad\text{かつ}\quad \underbrace{0 \le s}_{\text{②前方にある}} \quad\text{かつ}\quad \underbrace{s \le d_\mathrm{min}}_{\text{③壁より手前}}
```

②③ で「無限直線」を「今の反射区間の**線分**」に絞ったつもりでしたが、 これは**線分と球が交わる条件と同じではありません。**垂線の足が線分の外にあっても、 線分の端のほうが球の中に入っていることがあるからです。

> **例**：壁まで 1.0 m、受音球の中心が音線上 1.1 m、半径 0.2 m。
> 線分は 0.9〜1.0 m の区間で球の中にいるので**交わっています**が、
> $s = 1.1 > d_{\mathrm{min}} = 1.0$ なので ③ に引っかかって落ちていました。
> 同じことが $s < 0$（基点が球の中にいる）でも起こります。

どちらも**受音点が壁から受音半径以内にあるとき**に起きます。線分で判定すると 落としていた経路が拾えるようになり、判定は必ず**増える**方向にしか動きません （以前拾えていた経路が拾えなくなることはない）。`test.dxf` で受音点を天井から 0.08 m のところに置いて回すと、受音経路が **1590 → 1661 本**（**+4.5 %**）に増えました。

元コードのやり方は `method='foot'` として残してあります（**参照実装**）。

> **注意（書籍 図2.16, 2.17）**：受音半径は精度に直結します。
> 小さすぎる／音線数が少ないと、本来受音されるべき経路を取りこぼします（図2.16）。
> 大きすぎると、受音されないはずの経路まで拾ってしまいます（図2.17）。
> 本 PJ ではこの後 ⑤ の虚像法で経路を検証し直すので、多少の偽検出は救済されます。

5.7 鏡面反射

```
\boxed{\ \mathbf{v}' = \mathbf{v} - 2(\mathbf{v}\cdot\mathbf{n})\,\mathbf{n}\ }
```

意味：$(\mathbf{v}\cdot\mathbf{n})\mathbf{n}$ は、$\mathbf{v}$ を法線方向と接線方向に分解したときの **法線方向成分**です（$\mathbf{n}$ が単位ベクトルなのでこの形になります）。

`\mathbf{v} = \underbrace{(\mathbf{v}\cdot\mathbf{n})\mathbf{n}}_{\text{法線成分}} + \underbrace{\mathbf{v}_\parallel}_{\text{接線成分}}`

法線成分を 2 倍引くと、法線成分だけ符号が反転し、接線成分はそのまま残ります。

$$\mathbf{v}' = -(\mathbf{v}\cdot\mathbf{n})\mathbf{n} + \mathbf{v}_\parallel$$

これが鏡面反射（入射角＝反射角、書籍 図2.10）そのものです。壁に平行な動きは変わらず、 壁に垂直な動きだけが跳ね返る、と読めます。

反射後は基点を交点に移して次の反射に進みます。

$$\mathbf{p}_0 \leftarrow \mathbf{q}, \qquad \mathbf{v} \leftarrow \mathbf{v}'$$

対応コード: `sound_ray.reflection_generator()` , `sound_ray.soundraycomesfrom_renew()`

5.8 このループが記録するもの

音線法の段階では**エネルギーを計算しません**。記録するのは **「受音に至った経路が、どの壁面をどの順に反射したか」という壁面 ID の列**だけです。

Fortran では 2 つの配列を使い分けています。

Fortran	Python	役割
`tractmp(0:nref+1)`	`reflectionmeshid_history`	**一時**反射履歴。音線 1 本ごとに初期化し、反射のたびに壁 ID を追記
`traceff(count, 0:nref)`	`reflectionmeshid_history_2dim`	**受音した瞬間のスナップショット**を貯める 2 次元配列

重要な順序：元コードでは受音判定（663 行）が壁 ID の追記（677 行）**より前**にあります。 これは、受音した時点の履歴が「**そこまでに済んだ反射**」であるべきで、 これから当たる壁は含めてはいけないためです。受音と衝突は独立した条件で、

- 壁 ID の追記 → **衝突するたびに無条件**
- スナップショットの保存 → **受音するたび**

となります。この結論に沿って 2026-08-12 に `loop_reflectionmesh.py` を全面的に書き直しました（TODO A-1〜A-8 完了）。

⚠ 履歴の**先頭要素は番兵** `-1`（元コード `tractmp(0) = -1` ）で面 ID ではありません。 反射回数 = `len(history) - 1` です。下流で `mesh[history[k]]` と書くと `-1` が最後の面を指してしまうので、 必ず先頭を除いてから扱ってください（TODO A-9）。

5.9 可視化のための軌跡記録

対応コード: `ray_recorder.py`（新規モジュール）／詳細は `docs/出力・可視化方針.md`

音線追跡ループには、**本線とは別チャンネル**で軌跡を記録するフックが入っています。

```
loop(..., recorder=RayRecorder(total_rays=N, max_rays=2000))
```

本線が残すのは「受音した経路の反射面 ID 列」だけですが、可視化で見たいのは **音がどう広がるか**なので、受音しなかった音線も必要です。目的が違うので配列を分けています。 `recorder=None` なら一切動かないので、本番計算の速度・メモリには影響しません。

記録するのは音線 1 本につき、**節点座標・音源からの累積距離・反射面 ID・バンド別エネルギー・ 受音イベント・終了理由**です。ここから

- **音線の可視化** … `nodes` をそのまま折れ線として描く
- **音粒子の可視化（動画）** … `distances / c` が各節点への到達時刻なので、

  `position_at(t)` で任意時刻の粒子位置が線形補間で得られる

の両方が作れます。100 万本規模では全記録がメモリに乗らないため、 **絶対本数の上限＋等間隔ストライド**（既定 2000 本）で間引きます。

---

6. ④ 重複経路の削除

対応コード: `loop_deleteredundancy.py`（元コード 721〜841 行）

なぜ必要か：音線は N 本飛ばしますが、隣り合う音線は同じ壁面列を辿ることがよくあります。 壁 3 → 壁 7 → 壁 1 という経路が 50 本の音線から見つかっても、 **幾何学的にはそれは 1 本の経路**です。虚像法では経路 1 つにつき虚音源 1 組なので、 重複したまま渡すと同じ計算を 50 回やることになります。

書籍 式(2.66) のとおり、虚音源の総数は壁面数 m、最大反射回数 k に対して

$$N_{\text{虚音源}} = \sum_{i=1}^{k} m\,(m-1)^{i-1}$$

と**指数的に増えます**。全虚音源を総当たりするのが非現実的だからこそ音線法で候補を絞っているわけで、 その候補に重複が残っていては意味がありません。

Fortran は「反射回数でソート → 同一反射回数内で総当たり比較」という手順ですが、 Python では壁 ID 列をタプル化して `set` に入れるだけで等価な処理になります（順序は変わります）。

---

7. ⑤ 虚音源バックトレース

対応コード: `loop_noredundancy.py`（元コード 876〜1134 行） 書籍: 2.2.2 虚像法（p.86〜89）、図2.18、図2.19

音線法が見つけた壁面列に対して、**幾何学的に厳密な**到来時刻とエネルギーを計算し直す段階です。

7.1 虚音源をつくる（図2.18）

壁面に対する音源の**鏡像**を虚音源と呼びます。壁で反射する経路は、 「虚音源から受音点へ**まっすぐ飛んだ**経路」と幾何学的に等価です。これが虚像法の考え方です。

音源 $\mathbf{S}$ の、平面 $\mathbf{n}\cdot\mathbf{x}+d=0$ に関する鏡像は

$$\boxed{\ \mathbf{S}' = \mathbf{S} - 2\,(\mathbf{n}\cdot\mathbf{S} + d)\,\mathbf{n}\ }$$

意味：5.2 節のとおり $\mathbf{n}\cdot\mathbf{S}+d$ は**点 $\mathbf{S}$ から平面までの符号付き距離**です。 その距離の 2 倍だけ法線方向に動かせば、平面の反対側の対称な位置に着きます。 5.7 節の反射ベクトルの式と同じ構造（法線成分だけ反転）なのが見て取れます。

反射履歴に沿ってこれを繰り返し、$\mathbf{S}'_0=\mathbf{S}$（実音源）から $\mathbf{S}'_1, \mathbf{S}'_2, \ldots, \mathbf{S}'_k$ と虚音源の列を作ります。 $\mathbf{S}'_k$ は「k 回目の反射壁に対する、$\mathbf{S}'_{k-1}$ の鏡像」です。

★虚音源を目で見る（`view_images.py`。2026-08-24）

虚音源はコードの中だけの存在で、図2.18 を頭の中で組み立てないと追えません。 そこで**画面に描いて確かめられる**ようにしました（依頼 2026-08-24）。

★計算はやり直しません。パルス列（`結果/recN/<室>_pulses.csv`）から復元します。 到来方向 $\mathbf{e}$（受音点から虚音源を見込む単位ベクトル）と経路長 L があるので

$$\mathbf{S}'_k = \mathbf{x}_r + L\,\mathbf{e}$$

で虚音源そのものが出ます（`PulseList.image_sources`）。 虚音源と受音点を結ぶ**直線が「折れた経路を伸ばしたもの」**で、 その長さがそのまま経路長（= 音速 × 到来時刻）です。 **壁を貫いた線に見えるのが正しい姿**で、線が壁と交わる点が反射点にあたります。

検算として押さえていること（テスト [41]）：

見るもの	期待	実測
虚音源までの距離	経路長と一致	差 0（厳密）
直接音（0 回反射）の虚音源	**音源そのもの**	差 0
1 回反射の虚音源	壁について音源を鏡像にした点	差 0（6 面ぶん全部）

反射回数は**開始と終了**で絞ります（同じ値にすれば「N 回目のみ」）。 高次の虚音源は室の外へどこまでも離れる（研修室で経路長 3.4 km）ので、 既定は 0〜2 回で開き、`h`（室に寄る）と `r`（全部が入る）で行き来します。

★描いているのは**受音点に届いた経路の虚音源だけ**で、鏡像として作れる虚音源すべて ではありません（届かないものは 7.2 節でバックトレースが却下しています）。

7.2 経路の有効性を検証する（図2.19）

虚音源を作れても、その経路が実在するとは限りません。 壁は無限平面ではなく有限の三角形なので、鏡像がその壁の「外側」に対応してしまうことがあります。

書籍 p.88 の方法は、**受音点から逆向きに辿る**というものです。

1. 受音点 $\mathbf{x}_r$ から最後の虚音源 $\mathbf{S}'_k$ へ直線を引く
2. その直線が**最初に当たる壁**を求める（5.1〜5.5 とまったく同じ交差判定を使う）
3. それが履歴の k 番目の壁と**一致するか**を確認する
4. 一致すれば交点まで進み、$k-1$ について同じことを繰り返す
5. $k=0$（実音源）まで到達できれば、その経路は**有効**

途中で壁が食い違えば、その経路は現実には存在しないので破棄します。

★**「一致するか」は平面で見ます**（2026-10-21。元の作成者 豊田政弘氏の修正版 `backtrace_rev.f90` の改善 2 に合わせた）。虚音源の位置は**どの平面で映したか**だけで 決まり、その平面のどの部分（レイヤ・離れた板）で反射したかには依らないからです。 受音球には大きさがあるので、球の端を通って「壁の下部（レイヤ A）で反射」と記録された 経路を、球の中心から辿り直すと同じ壁の上部（レイヤ B）に当たることがあります。 経路としては正しいので通し、**吸音率は実際に当たったレイヤ（B）の値**で計算します。

それ以前は交差判定の単位（パッチ。吸音率を引くためにレイヤでも割ってある）で 照合していたので、この経路を捨てていました。hall1.dxf（音線 100 万本・反射 100 回）で 可視な虚音源が **61,667 → 73,817（1.20 倍）**、増えるのは主に 1 s 以降です。 JR の 3 室では、**表面積の 53〜63% がレイヤ違い・離れた部分に分かれた平面**の上にありました。

同じ平面の別レイヤで反射した履歴は同じ虚音源を作るので、照合の前に **反射履歴を平面の番号に直して重複を除きます**（`loop_noredundancy.plane_histories`）。 しないと同じ虚音源を 2 回数えます。

対応コード: `mesh_method.PatchArrays._planes()`（平面の番号 `plane_of_face`。 法線の向きが揃っていて、全頂点が相手の平面から 0.1 mm 以内なら同じ平面）／ `loop_noredundancy.backtrace_batch()` 書籍の例では、S_{12}（壁 1 → 壁 2 の順の虚音源）から受音点へ引いた直線が 壁 2 → 壁 1 の順で交わるので有効、S_{21} は順序が合わないので無効、と説明されています。

音線を戻す向きは、次の虚音源へ向け直します（元コード 1101〜1104 行）。

$$\mathbf{v} \leftarrow \frac{\mathbf{S}'_{k-1} - \mathbf{q}}{\|\mathbf{S}'_{k-1} - \mathbf{q}\|}$$

対応コード: `sound_ray.soundray_renew()`

7.3 反射によるエネルギー減衰 ← 本 PJ でいちばん重要な式

対応コード: `sound_ray.energy_decay()`（元コード 1091〜1094 行） 書籍: 式(2.58)〜(2.65)、図2.14

この式は書籍の**式(2.64)・式(2.65) そのもの**です。導出を追うと意味がはっきりします。

ステップ 1：斜入射の音圧反射係数（式2.58）

壁面の垂直入射比音響インピーダンス比を Z_n、入射角を θ とすると、 局所反応性（locally reacting）の壁面では

$$R = \frac{Z_\mathrm{n}\cos\theta - 1}{Z_\mathrm{n}\cos\theta + 1}$$

ステップ 2：吸音率とインピーダンスの関係（式2.61, 2.63）

吸音率は「入射したエネルギーのうち反射しなかった割合」なので、

$$\alpha = 1 - |R|^2$$

垂直入射（$\theta = 0$）のときの吸音率を α_n とすれば $|R_0| = \sqrt{1-\alpha_\mathrm{n}}$ となり、 これを式(2.58) に $\theta = 0$ で代入して Z_n について解くと

$$Z_\mathrm{n} = \frac{1+\sqrt{1-\alpha_\mathrm{n}}}{1-\sqrt{1-\alpha_\mathrm{n}}}$$

ステップ 3：代入して整理（式2.64）

これを式(2.58) に戻し、分母分子に $(1-\sqrt{1-\alpha_\mathrm{n}})$ を掛けて整理すると、

$$\boxed{\ R = \frac{\bigl(1+\sqrt{1-\alpha_\mathrm{n}}\bigr)\cos\theta - \bigl(1-\sqrt{1-\alpha_\mathrm{n}}\bigr)}{\bigl(1+\sqrt{1-\alpha_\mathrm{n}}\bigr)\cos\theta + \bigl(1-\sqrt{1-\alpha_\mathrm{n}}\bigr)}\ }$$

ステップ 4：エネルギーに直す（式2.65）

R は**音圧**の反射係数なので、エネルギーにするには 2 乗します。

$$E_\mathrm{r} = |R|^2 E_\mathrm{i}$$

コードでは $\cos\theta = |\mathbf{v}\cdot\mathbf{n}|$（どちらも単位ベクトル）で求め、 6 つの周波数バンドそれぞれについて累積エネルギーに掛けていきます。

この式が言っていること

- **入射角に依存する**。真正面から当たる（$\theta = 0$）と最もよく吸われ、

  掠めて当たる（$\theta \to 90°$、$\cos\theta \to 0$）と $|R| \to 1$ でほとんど吸われません。
  「浅い角度で入った音は壁で吸われにくい」という現実の挙動を表しています。
- $\theta = 0$ で $E_\mathrm{r}/E_\mathrm{i} = 1 - \alpha_\mathrm{n}$ になり、垂直入射吸音率の定義と一致します

  （実装の検証でも厳密に一致することを確認済み）。
- 入力は**垂直入射吸音率**である必要があります。

> ⚠ **実務上の注意（TODO D-7）**
> NEA で多く使われる**残響室法（乱入射）吸音率**は 1 を超えることがあり、そのまま入れると
> $\sqrt{1-\alpha}$ が虚数（コード上は NaN）になって計算が壊れます。
> 残響室法吸音率を使う場合は、Paris の式を逆に解いて垂直入射吸音率 α_n または
> Z_n を推定する前処理が必要です。
> 参考資料は `参考文献/01_吸音率・反射特性/垂直入射吸音率と乱入射吸音率.pdf` と、
> 書籍 3.1.2「吸音率測定」（抜粋 04）。

7.4 到来時刻

有効と判定された経路について、受音点と最終虚音源の距離から到来時刻を求めます。

$$r = \lVert \mathbf{x}_r - \mathbf{S}'_k \rVert, \qquad \boxed{\ t = \frac{r}{c}\ }$$

なぜ虚音源からの直線距離でよいのか：反射経路を鏡像で「まっすぐ伸ばした」のが虚音源なので、 $\mathbf{x}_r$ と $\mathbf{S}'_k$ を結ぶ直線の長さが、実際に音が壁を反射しながら辿った**折れ線の全長**と一致します。 これが虚像法の便利なところです。

7.5 この段階の出力

経路 1 本につき 1 行。元コードは 11 列のテキストです（元コード 1080 行）。

列	内容
1	反射回数 k
2	到来時刻 t [s]
3〜5	到来方向ベクトル $-\mathbf{v}_{\text{tgt}}$（受音点から見た音の来る向き）
6〜11	バンド別エネルギー $E_1 \ldots E_6$

これが**パルス列**（離散的な反射音の並び）です。次の段階でこれを連続的なインパルス応答に変換します。

Python 版（`PulseList`）は同じ内容を持ちますが、**到来方向を単位ベクトルにして距離を別列にしています。** 元コードの $-\mathbf{v}_{\text{tgt}}$ は正規化されておらず、その長さが経路長そのもの （＝「方向」と「距離」が 1 つの列に混ざっている）なので、分けたほうが 出力⑤（伝搬方向の可視化）にそのまま使えるためです。 `direction * distance` が元の値に一致します。

> **元コードとの相違点（鏡像の式）**：元コード 911 行は面までの距離に `abs()` を掛けていますが、
> 7.1 節の式は**符号付き**距離を使うのが正しい形です。両者が一致するのは虚音源が面の表側
> （$\mathbf{n}\cdot\mathbf{S}+d>0$）にあるときだけで、裏側にあると面を跨がずに
> **遠ざかる方向へ動いて**しまいます。凸な部屋では表側にしかならないので元コードでも
> 実害は出にくいのですが、凹んだ形状や法線を反転させたモデルでは崩れます。
> Python 版は符号付きで実装しています。

---

8. ⑥ インパルス応答の合成

対応コード: `impulse.py`（元コード `make_ipls_freq_monaural_fortran.f90`） 書籍: 式(2.67)、1.1.2 周波数分析

8.1 空気による減衰

音は空気中を伝わるだけでもエネルギーを失います（特に高音域）。 空気吸収減衰係数 m_{air} [1/m] を使って、

$$E' = E\exp\left(-m_{\text{air}}\, c\, t\right)$$

ct が伝搬距離なので、**距離に対して指数関数的に減衰**する形です。

m_{air} は **ISO 9613-1** で温度・湿度・気圧から計算します（`atmosphere.py`）。 **★引く周波数は厳密中心周波数**です（9.1 参照）。規格の表 1 も注 5 のとおり 厳密中心周波数で計算されているので、こうすると数字が表と一致します （20°C / 湿度 40% / 8 kHz バンドで **128 dB/km**。呼び値 8000 Hz のままだと 130 dB/km）。

中身は 2 つの機構の和です。

1. **古典吸収＋回転緩和** … 粘性と熱伝導による。周波数の 2 乗に比例
2. **酸素・窒素の振動緩和** … 分子が振動励起されてエネルギーを食う。

それぞれ緩和周波数を持ち、**湿度に強く依存**します（水分子が緩和を仲介するため）

実務上の効き方は、低音（〜250 Hz）ではほぼ無視でき、高音（4k〜8k Hz）では 100 m で 10 dB を超えることもある、という具合です。 **乾燥しているほど高音がよく減衰します**（緩和周波数が可聴域から外れ、吸収のピークが 可聴域に来るため）。意外に思われがちですが、これが規格どおりの挙動です。

> 元コードは $m_{\text{air}} = 1.81 \times 10^{-8} f^{1.57}$ という
> 「20°C・湿度 40%」を想定したべき乗近似で固定していました。
> 1k〜8k Hz では ISO と ±20% 程度で合いますが、**低音側は 1/3 程度しかありません**。

8.1.1 音速も温度・湿度から求める

音速も定数ではなく、湿り空気の状態方程式から計算します（`atmosphere.py`）。

$$c = \sqrt{\frac{\gamma R T}{M}}$$

なぜ湿度で変わるのか：水蒸気（M = 18.0 g/mol）は乾燥空気（28.96 g/mol）より **軽い**ので、湿気を含むほど平均モル質量 M が小さくなり、音速は**速くなります**。 比熱比 γ も水蒸気の混入でわずかに下がりますが、モル質量の効果のほうが大きくなります。

温度の効果のほうがずっと大きく、20°C 付近では **1°C あたり約 0.6 m/s**。

条件	音速
0°C / 0%	331.4 m/s
20°C / 0%	343.3 m/s
20°C / 40%（基準）	**343.8 m/s**
30°C / 60%	350.5 m/s

> 元コードの $c_0 = 340.0$ m/s は**およそ 14°C 相当**です。

音速はバックトレースの到来時刻、空気吸収の伝搬距離、音線軌跡の時間軸の**すべてに効く**ので、 `Atmosphere` オブジェクト 1 つを引き回して食い違いが起きないようにしています。

8.2 パルス列を周波数領域の伝達関数にする（式2.67）

n 番目のパルスのエネルギーを E_n、到来時刻を t_n とすると、伝達関数は

$$H(f) = \sum_{n=1}^{N} \frac{\sqrt{E_n}}{c\, t_n}\, e^{-j2\pi f t_n}$$

3 つの因子それぞれの意味：

- $\sqrt{E_n}$ … **エネルギーを音圧振幅に変換**しています。音の強さはエネルギー（振幅の 2 乗）で

  扱ってきましたが、波として足し合わせるには振幅に戻す必要があります。
- $\frac{1}{c\, t_n} = \frac{1}{r_n}$ … **球面波の距離減衰**。音圧は距離に反比例します

  （エネルギーは距離の 2 乗に反比例）。書籍 式(2.67) の $1/r_n$ そのものです。
- $e^{-j2\pi f t_n}$ … **時間遅れ t_n の周波数領域での表現**。時間領域で t_n だけ遅らせることは、

  周波数領域では各周波数成分の位相を $2\pi f t_n$ 回すことに相当します（時間シフト定理）。

つまりこの式は、「各反射音を、正しい大きさ・正しい遅れで重ね合わせる」ことを周波数領域で一気にやっています。

8.2.1 ★高速化：時間領域に置いてから FFT（2026-08-21／2026-08-23 検証）

式(2.67) をそのまま解くと、**周波数の数 × パルスの数**の掛け算になる。 実案件（研修室・パルス 8,825 本・応答 3 秒 → 周波数 70,313 点）で **15.5 秒**かかり、計算全体の 46% を占めていた。

同じものは**時間領域に置いてから FFT** しても得られる。

$$h_b(t) = \sum_{n} \frac{\sqrt{E_{n,b}}}{c\, t_n}\, \delta(t - t_n) \quad \xrightarrow{\text{FFT}} \quad H_b(f)$$

計算量は **パルスの数 ＋ FFT** になり、同じ条件で **0.24 秒（65 倍）**。

★★これは「時間領域で足す」ことではない（大事なところ）

査読論文 Toyoda & Sakayoshi (2021) の第 3 節は、まさにこのやり方を警告している。

> 古典的な虚音源法では、到来時刻と距離減衰を考えて**時間領域で**エネルギーを
> 積み上げてインパルス応答を得る。しかしディジタルのインパルス応答が要るときは、
> **サンプル間隔の間に届いたエネルギーをサンプル時刻へ寄せて**重ねなければならず、
> これが誤差になって、音圧ではなく**音のエネルギー**のインパルス応答になる。
> だから波動音響の結果と直接比べられない。そこで重ね合わせは周波数領域で行う（式(2)）。

問題になるのは「寄せる」ことで、時間領域そのものではない。 寄せると位相が最大で半サンプルずれるので、**干渉（くし形）の谷の位置が動く。**

こちらは寄せていない。到来時刻の**端数**を、窓付き sinc（Hann 窓）で 本来の位置に置いている。これは**理想遅延 $e^{-j2\pi ft}$ を帯域制限して 切り詰めたもの**なので、帯域内では位相ごと保たれる。

数字で確かめた（2 パルスの間隔をわざと 3.7135 サンプルにして谷の位置を見る）。

やり方	干渉の谷の周波数	11.3 kHz までの振幅差	位相差
厳密（式(2)）	5937.8 Hz	—	—
窓付き sinc（本実装）	**5937.8 Hz（一致）**	**0.00001 dB**	**0.00002°**
サンプル時刻へ寄せる	5512.5 Hz（**7% ずれる**）	4.6 dB	48.8°

タップ数を増やすと**単調に**厳密解へ寄る（＝式(2) を切り詰めているだけ、という証拠）。

タップ数	時間	厳密版の何倍速いか	実効値の比	波形の相関
64	0.21 s	78 倍	0.99744	0.99911
128	0.30 s	53 倍	0.99923	0.99970
256（既定）	**0.52 s**	**31 倍**	**0.99975**	**0.99988**
512	0.88 s	18 倍	0.99988	0.99996
厳密（式(2)）	15.9 s	1 倍	1	1

★足し込みに `np.bincount` を使っている（`np.add.at` は要素ごとに回るので遅い）。 結果は完全に同じで **256 タップで 1.9 倍・1024 タップで 3.7 倍**速くなった。 これで**タップ数を増やす負担が小さい**（＝速さと厳密さで悩まなくてよい）。

残る差は **8 kHz バンドより上**（12.7k・16k・20.2k の 1/3 オクターブバンド）に 集まるので、出している指標には出ない（T30 の差 0.0000 s、C50 の差 0.001 dB 未満）。

なおその 12.7k〜20.2k は、**入力（オクターブ 8 バンド。上端 8 kHz ＝ 上限 11.3 kHz） より上に、8 kHz バンドの値を写したもの**である（`band_mapping` が対数軸で いちばん近いオクターブバンドを選ぶため。低域側も 16〜50 Hz が 63 Hz の写し）。 **元コードの手書き対応表と同じ振る舞い**なので変えていないが、 入力の範囲より外に中身を作っていることになる（**TODO**。全エネルギーの 9.4%）。 高速版と厳密版の差はそのほとんどがここに乗っている。

一致は帯域制限つきなので、波形や伝達関数そのものを FDTD などの波動解と 突き合わせるときは厳密版を使う。設定画面の「インパルス応答の合成」で **厳密（式(2) をそのまま）**を選べる（`project.json` の `impulse_method`）。 コードでは `impulse.transfer_function` / `method="exact"`。

8.3 バンド分割・畳み込み・逆変換

バンド別の吸音率を反映させるため、1/3 オクターブ 32 バンドに分けて処理します。

1. **バンドパスフィルタの生成** … 各バンドの通過帯域を持つ FIR フィルタ h_t を作る

   （sinc 関数の差に Hamming 窓を掛けたもの）
2. **フィルタを FFT** … $h_t \to H_t(f)$
3. **負の周波数側を複素共役で拡張** … 実数信号の FFT は $H(-f) = H^*(f)$ を満たす必要があるため
4. **周波数領域で乗算** … $H(f) \cdot H_t(f)$。

   周波数領域での掛け算は、時間領域での畳み込みと等価（畳み込み定理）。
   時間領域で畳み込むより計算量が大幅に少なくて済みます。
5. **逆 FFT して 32 バンドを合算** … 各バンドの時間応答を足し合わせて 1 本のインパルス応答にする
6. **CSV 出力** … 時間ベクトルを付けて保存

> **注意（書籍 p.86）**：ここで得られるのは**エネルギーの応答**を音圧に直したものであり、
> 波動論的に厳密な音圧のインパルス応答とは異なります。位相情報は反射経路の遅延のみに由来し、
> 回折や壁面での位相変化は考慮されていません。

8.4 バンドパスフィルタの遅れと、周波数領域の掛け算の落とし穴

上の 1. で作る FIR フィルタは**左右対称（直線位相）**です。対称なフィルタは 「入力より前の時刻にも応答がある」非因果的なものなので、実際に使うときは 中央がタップの真ん中に来るようにずらします。その結果、**出力全体が $(N-1)/2$ サンプルだけ遅れます**（N = タップ数）。

元コードは N = データ点数 nn = 131072 としているため、遅れは **1.49 秒**にもなり、 しかも補正していないので、出力される CSV は 「0 秒付近は無音で、1.49 秒あたりから音が始まる」形になっていました。

さらに、周波数領域で掛け算するということは**巡回畳み込み**をしているということです。 巡回畳み込みでは、フィルタの「負の時刻側の応答（プリリンギング）」が **バッファの末尾に回り込みます**。これは実在しない作り物の音です。

Python 版はどちらも次のように解決しています。

- **ゼロ詰めしてから掛ける**ことで線形畳み込みにし、回り込みを起こさない
- フィルタ長を実用的な長さ（既定 8192 タップ ≒ 93 ms）にし、

  遅れぶんを切り落として返す

結果として、**出力されるインパルス応答は 0 秒が音の始まり**になります。

> この回り込みは残響時間（9 章）でさらに深刻な影響を及ぼします。
> 減衰曲線に -20 dB の床を作って T_{30} を測れなくしてしまうためです。詳細は 9.2 節。

---

9. ⑦ 残響時間の算出

対応コード: `reverberation.py`（元コード `ipls2rt_fortran.f90`） 書籍: 2.3.3「残響式」（Sabine 式・Eyring 式）

インパルス応答から残響時間 T_{60}（音圧レベルが 60 dB 減衰するまでの時間）を求める段階です。

9.1 オクターブバンドに分ける

残響時間は周波数ごとに違うので、まずオクターブバンド（既定は 63〜8000 Hz の 8 バンド）に分けます。

バンドの決め方は **IEC 61260-1:2014（JIS C 1513-1:2020）** に従います。ここで **「呼び中心周波数」と「厳密中心周波数」は別物**である点が要点です。

	何に使うか	**値**
呼び中心周波数（3.6）	表・図・ファイル名	63 / 125 / 250 / … / 8000
厳密中心周波数（5.4.1 式(2)）	**計算**	$f_m = f_r G^{x/b}$

$$f_r = 1000\ extHz¤¤¤¤¤¤¤5.3¤, \qquad G = 10^{3/10} = 1.99526\ ext¤5.2.1¤$$

たとえば呼び値 8000 Hz のバンドの実体は **7943.3 Hz** です。帯域の上下端は

$$f_1 = f_m\, G^{-1/(2b)}, \qquad f_2 = f_m\, G^{+1/(2b)} \qquad ext¤5.6.1¤(4)(5)¤$$

で、オクターブ（$b=1$）なら f_m の 0.7079 倍と 1.4125 倍になります。

> ★`1/\sqrt2` **倍・** `\sqrt2` **倍ではありません。** それは $G=2$ とする
> 「ベース 2 フィルタ」の値で、規格 5.2.2 注記 2 が
> 「適合する確率は中心周波数が基準周波数から離れるほど下がる」としている側です。
> 差は 0.12% と小さいものの、規格に沿うほうが説明できるので
> **2026-09-16 にベース 10 へ揃えました**（ `absorption.band_edges` ）。
> 1/3 オクターブの 0.89125/1.12202 は規格 表 F.1 の不連続点の値そのものです。

フィルタは **IEC 61260**（オクターブフィルタの規格。ISO 3382 の残響測定が前提にしている） の**中心周波数と帯域端**に合わせた **Butterworth 6 次**を使います （ `scipy.signal.butter` + `sosfilt` ）。

> **「クラス 1」とは名乗りません。** IEC 61260 のクラスはフィルタの次数ではなく、

> **減衰量マスク（通過域・遷移域・阻止域の許容範囲）への適合**で決まるものです。
> その試験は本 PJ では行っていないので、規格の中心周波数に基づく Butterworth である、
> という言い方に留めています（2026-09-05）。

9.2 なぜ元コードのフィルタでは測れないのか ← つまずきポイント

元コードは 8.4 節と同じ**巡回畳み込み＋タップ数 = データ点数**の FIR を使っています。 残響時間の算出では、これが致命的になります。

プリリンギングがバッファ末尾に回り込むと、そこに**実在しないエネルギー**が乗ります。 Schroeder 積分（次節）は「時刻 t 以降に残っているエネルギー」を見るものなので、 末尾に偽のエネルギーがあると、減衰曲線が途中で**下げ止まって床を作ります。**

実際に減衰率が既知の応答で試すと、床は -20 dB あたりにでき、 評価に必要な -35 dB まで落ちません。その結果 T_{30} は **理論値の 5〜19 倍**になるか、そもそも算出できませんでした。

Butterworth のような**因果 IIR フィルタ**にすると、そもそも「入力より前の応答」が 存在しないので、この問題は構造的に起きません。

9.3 Schroeder 積分（逆方向積分）← この段階の核心

インパルス応答の 2 乗をそのまま dB 表示すると、反射音の位相がランダムに干渉するため グラフが激しく暴れて傾きが読めません。そこで**後ろから積分**します。

$$\boxed{\ D(t) = \int_t^{\infty} h^2(\tau)\,d\tau\ }$$

意味：「時刻 t 以降に**まだ残っているエネルギー**」です。 積分（＝足し算）するとランダムな揺らぎがならされ、滑らかな減衰曲線になります。 これを最大値で規格化して dB 表示したものが**減衰曲線**です。

$$L(t) = 10\log_{10}\frac{D(t)}{D(0)}$$

Schroeder が示したのは、「1 本のインパルス応答をこう積分したものが、 **多数回の残響測定を平均した結果と一致する**」ということです。1 回の計算で滑らかな曲線が得られます。

9.4 傾きから T_{60} を外挿する — EDT / T20 / T30

60 dB 減衰を最後まで測ることは実務ではほとんどありません。 暗騒音に埋もれるうえ、そこまでの直線性も保証されないためです。 そこで減衰曲線の**直線な部分だけを測って 60 dB 相当に外挿**します。

ISO 3382 が求めているのは、評価区間の全サンプルへ直線を最小二乗で当てることです。 その傾き a [dB/s] から

$$\boxed{\ T = -\frac{60}{a}\ ,\qquad \sum_i \left(L(t_i) - (a\,t_i + b)\right)^2 \rightarrow \mathrm{min}\ }$$

$L(t_i)$ は減衰曲線、和は評価区間に入る**全サンプル**についてです。

区間の取り方で名前が変わります。

指標	評価区間	性質
EDT（初期減衰時間）	0 〜 -10 dB	**聴感上の響きの短さに近い**とされる。初期反射の密度を反映するので、後部残響が同じでも室形状で差が出る
T20	-5 〜 -25 dB	暗騒音が高い実測でも取りやすい
T30	-5 〜 -35 dB	最も一般的（ISO 3382 の標準）。元コードもこれ

-5 dB から始めるのは、**直接音の直後は曲がりやすい**ためです（直接音と初期反射が 密に来るので、統計的な減衰から外れる）。

`reverberation.decay_measures()` が 3 つまとめて返します。 減衰曲線は 1 回作ればよく、評価区間を変えて読み取るだけです。

傾きから求めるので、応答全体が一定量ずれていても影響を受けません。

2 点法との違い ← 2026-09-05 に変更したところ

2026-09-05 より前は、**開始 dB と終了 dB を横切る 2 点の時刻差**から求めていました （元コード `ipls2rt_fortran.f90` 134 行のやり方）。

$$T_{\text{2点法}} = \big(t_{\text{終了dB}} - t_{\text{開始dB}}\big)\times \frac{60}{\text{開始dB} - \text{終了dB}}$$

- **減衰がまっすぐなら両者はほぼ一致します**（合成した指数減衰で差 1 % 未満）。

  それ以前に出した数字が的外れだった、という意味ではありません
- **曲がっていると大きくずれます。** `test.dxf`（2×3×1 m）の実測で、

  125 Hz バンドの T_{30} が 0.135 s（2 点法）に対し 0.184 s（回帰）——**35.7 % 違い**ました
- 回帰にすると**適合の良し悪しが数値で出る**のが実利です（次節）

2 点法は `fit='crossing'` として残してあります（元 Fortran との一致確認に使うため）。

9.4.1 曲率と非線形性 — 結果を信用してよいかの目安

減衰が直線でなければ T_{30} に意味はありません。**2 つの数字**で見ます。

① 曲率 C（ISO 3382-1） — T_{20} と T_{30} の食い違い。

$$C = \left(\frac{T_{30}}{T_{20}} - 1\right) \times 100\ \%$$

0 に近ければ直線。**10 % を超えたら疑う。** T_{20} は $-5 \sim -25$ dB、T_{30} は $-5 \sim -35$ dB の区間で読みます。 **どの区間で残響時間を出したかに依らず、この 2 つで測ります**—— C は減衰曲線そのものの性質だからです。

② 非線形性 ξ（ISO 3382-2） — 回帰直線への乗り具合。

$$\xi = 1000\,(1 - r^2)$$

r は回帰の相関係数です。**回帰にしたことで無料で手に入る指標**で、 小さいほど直線に乗っています（目安は `\xi \le 10`）。 曲率が「2 つの推定値のずれ」を見るのに対し、ξ は「区間の中の全点のばらつき」を見るので、 **同じ曲がり方でもこちらのほうが早く気づけます。**

どちらも `結果/<室>_rt.csv` に `curvature_percent` / `<指標>_xi` として残ります。

③ 減衰の余裕 — ISO 3382 は評価区間の下端より**さらに 10 dB 下**まで 減衰が見えていることを求めます（T_{30} なら -45 dB）。 実測では暗騒音の話ですが、シミュレーションでは **最大反射回数の打ち切り**と**インパルス応答長の打ち切り**が同じ役目の「床」を作ります。 届いていないバンドは端末に知らせ、`decay_floor_db` として CSV にも残します。

本 PJ でいちばんよくある原因は**最大反射回数 `nref` の不足**です （2026-10-30 から、自動にすれば応答の長さまで届く回数を試し追跡で決めます。4.3 節）。 エネルギーが 35 dB 減衰する前に音線追跡を打ち切ると、 残響が途中で途切れて曲率が大きく出ます。

必要な反射回数の目安は、平均自由行程 $\bar{\ell} = 4V/S$ と吸音率から

$$n \approx \frac{3.5}{-\log_{10}(1-\alpha)}$$

例えば $\alpha = 0.05$ なら 157 回。**吸音率が小さい部屋ほど大量の反射が要ります。**

9.6 統計残響式（Sabine / Eyring / Eyring-Knudsen）← 物差しになる

対応コード: `reverberation.statistical_reverberation()` 書籍: 2.3.3「残響式」

音線を飛ばさず、室容積 V と各面の面積・吸音率だけから残響時間を見積もれます。 拡散音場（音がどの方向からも等確率に来る）を前提にした古典的な式で、 シミュレーション結果が妥当かを確かめる物差しになります。

> **閉じた室が前提です。** 開いた形状（一面だけの壁など）では容積が定義できません。

共通の形

$$\boxed{\ T_{60} = \frac{24\ln 10\;V}{c\,A}\ }$$

A は**等価吸音面積**（単位 m²、metric sabin）。 $24\ln 10/c$ は $c = 343$ m/s のとき **0.161** になり、よく見る $0.161V/A$ の形になります。 本 PJ では音速を大気条件から求めるので、0.161 に固定していません。

なぜ V/A の形なのか：1 秒あたりの反射回数は「音速 ÷ 平均自由行程」 $= cS/(4V)$。 1 回の反射でエネルギーが $(1-\alpha)$ 倍になるので、減衰の速さは $cS\alpha/(4V)$ に比例します。 その逆数が残響時間のスケールで、60 dB ぶんの係数を掛けると上の式になります。

3 つの式の違い — A の作り方

式	等価吸音面積 A	向いている場面
Sabine	$S\bar{\alpha}$	最も古典的。**吸音率が小さいとき**（〜0.2）。$\bar{\alpha} \to 1$ でも T が 0 にならない欠点がある
Eyring	$-S\ln(1-\bar{\alpha})$	反射のたびに $(1-\bar{\alpha})$ 倍になると考える。**吸音率が大きいとき**
Eyring-Knudsen	$-S\ln(1-\bar{\alpha}) + 4mV$	アイリングに**空気吸収**を足したもの。高音域で効く

$\bar{\alpha} = \sum_i S_i\alpha_i / S$ は面積で重み付けした平均吸音率です。

Eyring が Sabine より短く出る理由： $-\ln(1-\bar{\alpha}) > \bar{\alpha}$ だからです。 $\bar{\alpha}$ が小さいときは両者はほぼ同じですが、大きくなるほど差が開きます。

★アイリングとアイリング・ヌードセンを分けて出す

アイリングの式そのものは $-S\ln(1-\bar{\alpha})$ までで、 **空気吸収の項 $4mV$ を足した形はヌードセン（V. O. Knudsen）の寄与**です。

そこで本 PJ は**両方を別の列で出します**。差がそのまま**空気吸収の効き**になるので、「この室で空気吸収がどれくらい効いているか」がそのまま読めます。 $4mV$ の m はエネルギーの減衰係数 [1/m]（8.1 節）で、高音域ほど大きくなります。

Sabine と Eyring には空気吸収を入れていません。 素の古典的な形にしておかないと、 Eyring → Eyring-Knudsen の差が「空気吸収ぶん」にならないためです。

研修室での実際の値（V = 372.96 m³）：

	125Hz	250Hz	500Hz	1kHz	2kHz	4kHz
Sabine	1.260	0.859	0.494	0.332	0.360	0.336
Eyring	1.181	0.779	0.412	0.247	0.276	0.250
Eyring-Knudsen	1.177	0.775	0.410	0.245	0.271	0.238

空気吸収の効きは 125 Hz で 0.004 秒、4 kHz で 0.012 秒。 **この室では小さい**（容積が小さく残響も短いため）。大空間ほど効いてきます。

ミリントンの式について（採用していません）

ミリントン・セッテの式（Millington 1932 / Sette 1933）という式もあります。 アイリングが**平均してから対数を取る**のに対し、**面ごとに対数を取って足し合わせ**ます。

$$A = -\sum_i S_i \ln(1-\alpha_i)$$

吸音率が面ごとに大きく違うときの理屈としては通っていますが、**致命的な癖**があります。 $\alpha_i \to 1$ の面が 1 枚でもあると $-\ln(1-\alpha_i) \to \infty$ となり、 A が発散して $T \to 0$ になります。「開口（$\alpha = 1$）が 1 つあるだけで 室の残響がゼロ」という結論になってしまうため、**実務では使われません。**

一時実装していましたが、研修室（吸音面 $\alpha_s = 0.951$ が 127 m²）で 中高域が Eyring-Knudsen の約半分に出たこともあり、**2026-08-17 に外しました。**

★注意：使う吸音率は「乱入射」

これらの式は拡散音場が前提なので、**乱入射吸音率** α_s を使います。 本 PJ の `Mesh` が持っているのは反射計算用の**垂直入射**吸音率 α_n なので、 Paris の式（9.5 節）で変換してから代入します（ `convert_to_random=True` が既定）。

変換しないと吸音を過小評価し、**残響時間が長く出ます。**

突き合わせの読み方

シミュレーションと統計式が食い違っても、**どちらが正しいという話ではありません。**

- 統計式 … 拡散音場を仮定した**平均像**。受音点の位置を問わない
- シミュレーション … **特定の受音点**での実際の減衰。鏡面反射のみ

大きく食い違うときは、音場が拡散していない（小さい室・平行面・吸音率の偏り）か、 設定に問題があるかの手がかりになります。

9.5 吸音率の「種類」← 実務でいちばん間違えやすいところ

対応コード: `absorption.py`

カタログに載っている吸音率には少なくとも 2 種類あり、**混同すると結果が大きく狂います。**

種類	記号	測り方	特徴
垂直入射吸音率	α_n	音響管（インピーダンス管）	正面から当てた場合だけ。1 を超えない
残響室法（乱入射）吸音率	α_s	残響室	**1 を超えることがある**

本 PJ の反射計算（7.3 節・式2.64）が要求するのは α_n。 α_s をそのまま入れると $\sqrt{1-\alpha}$ が NaN になります。

9.5.0 吸音率はどこから来るか ― 条件表（2026-08-21 に変えた）

以前は **CAD のレイヤ名**が材料名で、レイヤ名を書き換えないと材料を変えられませんでした。 壁の吸音材を差し替えるたびに CAD を直して読み込み直すのは現実的でないので、 **「条件表」（`プロジェクトフォルダ/条件表.xlsx`）で決める**形に変えました。

シート	中身	誰が書くか
「吸音率」	PJ で使う材料の一覧（番号・材料名・α・**種類**）。1000 材料の枠	利用者
その他のシート（1 枚 = 1 条件）	レイヤー名ごとに**材料番号**と**安全率**だけ書く。**シート名が条件名**になり結果ファイル名に入る	利用者

決めごと：

- **★入力は「材料番号」と「安全率」の列だけ。**材料名は全角・半角のミスタイプを

誘発するので入力に使いません（材料名の列は `VLOOKUP` の参考で、読むときは見ない）

- **★種類（残響室法／垂直入射）はドロップダウンで選ぶ。**取り違えると吸音を

大きく誤るため（この節の主題）。α_s なら Paris の式で自動変換されます

- **★安全率**（例 0.8）は「吸音率を見過ぎないように間引く」係数。未入力なら掛けません。

★カタログ値に掛けてから垂直入射へ変換する（Paris の式は非線形なので順番で意味が変わる）

- 条件を並べて**一括計算**でき、最後に**条件を横に並べた比較表**を作ります
- 面ごとに材料を貼ることもできます（`face_editor.py`。1 つの 3DSOLID で出来ていて

レイヤが分かれていないモデル用の逃げ道）

対応コード: `condition_table.py`（`absorption_table()` が上の順番どおりに掛ける）

9.5.1 Paris の式

規格化音響インピーダンス z（実数と仮定）が分かれば、両者は結び付きます。

$$\alpha_n = 1 - \left|\frac{z-1}{z+1}\right|^2 = \frac{4z}{(1+z)^2}$$

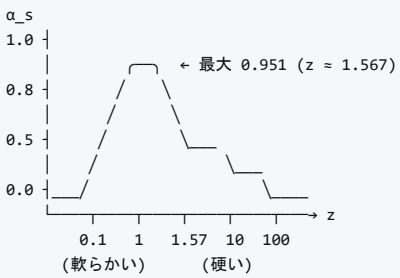
```
\alpha_s = 2\int_0^{\pi/2}\alpha(\theta)\sin\theta\cos\theta\,d\theta = \frac{8}{z^2}\Big[(1+z) - 2\ln(1+z) - \frac{1}{1+z}\Big]
```

α_s の重み $2\sin\theta\cos\theta$ は「立体角の重み $\sin\theta$」と「面に投影したときの割合 $\cos\theta$」の積で、全体の積分が 1 になるよう規格化されています。**あらゆる方向から等確率に音が来る（拡散音場）としたときの平均**という意味です。

9.5.2 逆に解くときの注意

α_s から z を求めるのが実務で必要な向きですが、**素直には解けません。**

α_s は z について**単調ではない**からです。$z \to \infty$（硬い壁）でも $z \to 0$（完全に軟らかい壁）でも $\alpha_s \to 0$ になり、その間の $z \approx 1.567$ で最大値 **0.951** を取ります。つまり 1 つの α_s に対して z が 2 つあります。



建築材料は空気より音響インピーダンスが高いので、`z \ge 1.567` **の枝**を選びます。また 0.951 を超える α_s（残響室法では 1 を超えることすらある）は局所反応モデルでは表せないので、上限に丸めて警告します。

9.5.3 どれくらい違うのか

残響室法 α_s	z	垂直入射 α_n
0.20	32.6	0.116
0.50	9.66	**0.340**
0.80	3.88	0.652

残響室法の値をそのまま垂直入射として使うと、吸音を大幅に過大評価します。$\alpha_s = 0.50$ の材料を $\alpha_n = 0.50$ として入れると、実際の 0.34 より 1.5 倍吸音することになり、**残響時間が短く出ます。**

吸音率 CSV には種類を宣言する行（`# kind: random`）を書いてください。

9.7 経路差から見たモード ― 吸音の効果を目で見る

低い周波数では、室の中の音は「どこにでも同じように広がる」のではなく、 **特定の周波数だけが強く残る**（固有振動＝モード）。 書籍 2.1 の波動音響理論が扱う世界です。幾何音響（音線法）は本来この世界を 苦手としますが、**虚音源の並びは固有振動と表裏の関係**にあるので、 パルス列の足し方を変えるだけで低域のモードを覗くことができます。

9.7.1 経路差が波長の整数倍になると強め合う

受音点には、直接音のあとに反射音が次々に届きます。 到来時刻の差 Δt に音速を掛けたものが**経路差** $\Delta = c\,\Delta t$ です。

同じ音が経路差 Δ だけ遅れて重なると、 Δ が波長の整数倍のときに山と山が揃って**強め合い**、 半波長ずれていると**打ち消し合い**ます。強め合う周波数は

$$f = m\frac{c}{\Delta}, \qquad m = 1, 2, 3, \ldots$$

これが固有周波数と一致します。 たとえば直方体の x 方向を往復する音線は $\Delta = 2L_x$ なので $f = m\,c/(2L_x)$ となり、軸モードの式

$$f_{m,0,0} = \frac{c}{2}\sqrt{\left(\frac{m}{L_x}\right)^2} = \frac{mc}{2L_x}$$

そのものです。つまり**経路差を数えれば、どの固有振動が何本の経路で 支えられているかが分かります**。

9.7.2 位相込みで足すと「重なった本数」になる

パルス列を、振幅 a_n・到来時刻 t_n として位相込みで足します。

$$H(f) = \sum_n a_n\, e^{-j2\pi f t_n}$$

指数の中身が経路差ぶんの位相です。ここで $a_n = 1$（**減衰をいっさい考えない**） とすると、$|H(f)|$ は

> **その周波数で同位相に重なった経路の本数**

そのものになります。1 本なら 1、2 本重なれば 2、3 本重なれば 3。 向きがばらばらならランダムウォークになって $\sqrt{N}$ 程度にしかなりません （N は全経路数)。図にはこの $\sqrt{N}$ の線を引いてあり、 **この線を超えている山だけが「本当に強め合っている周波数」**です。

9.7.3 吸音の効果を差として見る

同じ足し算で、重みだけを変えます。

重み a_n	意味
1	減衰なし。**室形状だけ**で決まる。吸音材を貼っても変わらない
$1/d_n$	**完全反射**。距離減衰だけ。壁は 100% 跳ね返る
$\sqrt{E_n}/d_n$	**設計の吸音あり**。E_n は各面の吸音率を掛けたエネルギー

後ろ 2 本の差が、**その周波数で吸音材が削っている量**です。

これができるのは、バックトレースが記録するパルスのエネルギー E_n に **面の吸音しか入っていない**からです（距離減衰 $1/(ct)$ と空気吸収は 式(2.67) の段階で掛ける約束。8.2 節)。だから後から自由に組み替えられます。

空気吸収はこの図では入れていません。**この帯域では効かない**ためです （ISO 9613-1、20°C・湿度 40% で 125 Hz なら 100 m 進んで 0.05 dB)。

9.7.4 計算量は増えない

素直に書くと「パルス本数 × 周波数点数」の掛け算になり、 10 万本 × 600 点で 6000 万回と重くなります。しかし到来時刻を細かい格子に **投げ込んで（ヒストグラムにして）から FFT** すれば $O(n \log n)$ で済みます。 やっていることはインパルス応答を作る手順（8 章）と同じです。 重みを変えて 3 回描いても一瞬で終わります。

9.7.5 検算 ― モード形状まで再現できている

平行 2 面（間隔 $L = 4$ m、音源 $x = 1$ m、受音点 $x = 3$ m）で試すと、 軸モード 42.9 / 128.6 / 171.5 Hz が山として出ます。

2 次の 85.8 Hz だけは出ません。 これは誤りではなく、 モードの形 $\cos(m\pi x/L)$ が $m = 2$ のとき音源・受音点の両方でちょうど 0、 つまり**節に乗っている**からです。節に音源を置けばそのモードは励振されず、 節で聞けばそのモードは聞こえません。

$$\cos\frac{2\pi\cdot 1}{4}\cdot\cos\frac{2\pi\cdot 3}{4} = 0$$

固有周波数だけでなくモードの形まで再現できていることの確認になっています。

9.7.6 周波数の刻みは 1 Hz にまとめる

FFT の周波数刻みは応答長の逆数で決まってしまいます（3 秒なら 0.333 Hz）。 細かすぎて線がぎざぎざし、かえって読みにくいので、**1 Hz 幅にまとめ直して**います。

このとき、**点を間引くのではなく二乗平均します。**

$$|H|_{1\,\mathrm{Hz}} = \sqrt{\frac{1}{K}\sum_{k}|H(f_k)|^2}$$

減衰を考えない①は、減衰が無いぶん櫛の歯が鋭く、**その幅は応答長でしか 決まりません**（3 秒なら 0.333 Hz）。点を飛ばすと山を跨いで見落とします。二乗平均なら「その 1 Hz の中にどれだけ入っているか」になるので取りこぼしません。

もうひとつ利点があります。位相がばらばらのときの目安 $\sqrt{N}$ は **帯域幅に依らず保たれる**ので、まとめ方を変えても図の基準線を引き直さずに済みます。

9.7.7 読むときの注意

- 0 Hz では位相差が消えるので全経路が同位相になります。左端に巨大な山が

  立ちますがこれは当たり前で、**いちばん低い固有周波数より下は描いていません**
- 吸音率はオクターブバンドの値しか無いので、各周波数点には**最も近いバンド**の値を

  当てています。6 バンド（125 Hz 始まり）だと 88 Hz より下はすべて 125 Hz の値です。
  低域を細かく見たいときは 8 バンドにしてください
- パルス列は受音球が拾えた経路の**標本**です。本数の絶対値は音線数と受音球の半径で

  変わります。読むべきは**山と谷の位置**と、**2 本の線の差**です

9.8 音圧レベル — 絶対値を出す（依頼 2026-08-21）

> 音圧レベルを知りたい。無論、帯域毎に必要。例えば、無響室の計算で
> 逆二乗がどれくらい成り立っているかを確認したりしたい。
> 音源に点音源の PWL を与えれば、絶対値もわかればよいですが、
> 未入力の場合、相対値でも良いです。

9.8.1 パルス列には距離減衰が入っていない

バックトレースが出すパルス列（§7.5）のエネルギー A_i には、 **反射での吸音だけ**が入っている。距離減衰と空気吸収は インパルス応答を合成する段階（§8.1・§8.2）で掛けている。

そこで「受音点で実際に受け取るエネルギー」を 1 か所で作る。

$$E_i = A_i\, e^{-m_{\mathrm{air}} d_i}\, \frac{1}{4\pi d_i^2}$$

- A_i … パルス列のエネルギー（反射の吸音の積）
- $e^{-m d_i}$ … 空気吸収（§8.1 と同じ m）
- $1/(4\pi d_i^2)$ … 点音源の球面拡散

$\sqrt{E_i}$ が §8.2 の伝達関数の振幅 $\sqrt{A_i}/(c t_i)$ に $1/\sqrt{4\pi}$ を 掛けたものになっている。**つまりインパルス応答と同じ量をエネルギーのまま書いたもの**で、4π はインパルス応答では定数として省いてよかったが、絶対値には要る。

9.8.2 音圧レベル

$$L_p = L_W + 10\log_{10}\left(\sum_i E_i\right) + 10\log_{10}\frac{\rho c}{400}$$

最後の項は空気の特性インピーダンス ρc が $400\ \mathrm{N\,s/m^3}$ から ずれる分の補正で、20°C / 湿度 40% / 101.325 kPa なら $\rho c = 412.5$、 補正は **+0.13 dB**。密度は湿り空気の状態方程式から出す（`atmosphere.density`）。

$$\rho = \frac{p M}{R T}, \qquad M = (1 - x_w) M_{\mathrm{dry}} + x_w M_{\mathrm{water}}$$

9.8.3 ★逆二乗則が厳密に成り立つことの確認

反射面が無ければパルスは直接音 1 本だけで $\sum E_i = 1/(4\pi d^2)$、したがって

$$L_p = L_W - 10\log_{10}(4\pi d^2) + 10\log_{10}\frac{\rho c}{400} = L_W - 20\log_{10} d - 10.99 + 0.13$$

教科書の $L_p = L_W - 20\log_{10} r - 11$ と一致する（-11 は $10\log_{10} 4\pi = 10.99$ を丸めた値）。

これは実装の物差しになる。無響室（反射面なし）のモデルで計算すると 直接音が理論値に一致し、距離を 2 倍にすると 6.02 dB 下がる。 反射面があればそのぶん上に外れ、外れ量がそのまま**室の効き**になる。 出力（`spl.csv` / `spl.png`）には自由音場の理論値と**その差**を必ず並べてある。

L_W が未入力のときは $L_W = 0$ dB（＝音響出力 1 pW）として計算する。 帯域ごとの相対関係と距離依存性はそのまま読めるので、逆二乗の確認には足りる。

9.8.4 成分に分けて出す

行	中身	使いどころ
L_p	全パルスの和	その点の音圧レベル
直接音	いちばん早く届くパルスだけ	自由音場との比較
反射音	残り全部	室の効き
初期 / 後期	直接音から 50 ms で分ける	C50 と同じ切り方
A 特性	IEC 61672-1 の補正を掛けたもの	騒音の評価

9.9 STI（音声伝送指数）← 依頼 2026-08-21

IEC 60268-16。「音声の**変調**がどれだけ保たれて届くか」で明瞭度を測る。 残響や反響は変調（音の大小の起伏）を鈍らせるので、その鈍り方を指標にする。

9.9.1 変調伝達関数 $m(F)$

```
m(F) = \frac{\left|\displaystyle\int_0^\infty h^2(t)\,e^{-j2\pi F t}\,dt\right|} {\displaystyle\int_0^\infty h^2(t)\,dt}
```

$h^2(t)$ は**エネルギー的インパルス応答**なので、 **パルス列の（到来時刻, 受け取るエネルギー）がそのまま使える。**

$$m(F) = \frac{\left|\sum_i E_i\, e^{-j2\pi F t_i}\right|}{\sum_i E_i}$$

帯域フィルタを通した波形を数値積分するより素直で、フィルタの遅れや裾の影響を受けない。 幾何音響では普通この形で出す。変調周波数 F は 0.63〜12.5 Hz の 14 個 （1/3 オクターブ間隔）、オクターブバンドは 125 Hz〜8 kHz の 7 帯域。

検算：指数減衰 $h^2(t) = e^{-13.8t/T}$ なら解析的に

$$m(F) = \frac{1}{\sqrt{1 + \left(\frac{2\pi F T}{13.8}\right)^2}}$$

となる。実装はこれと 10^{-6} 以下で一致する（`tests/test_geosim.py` [32]）。

9.9.2 背景騒音と聴覚マスキング

$$m'(F) = m(F) \cdot \frac{I_{\text{signal}}}{I_{\text{signal}} + I_{\text{noise}} + I_{\text{masking}} + I_{\text{threshold}}}$$

マスキングは下の帯域から上の帯域へ掛かり、その量は下の帯域のレベルで決まる。 受聴閾値 $I_{\text{threshold}}$ は IEC の表（125 Hz で 46 dB、1 kHz で 6.5 dB）。

★絶対値が要るので、音源 PWL と騒音レベルの**両方**が入力されているときだけ 効かせる。片方でも無ければ「騒音なし・理想の SNR」として計算する（そのぶん STI は高めに出るので、出力にどちらで計算したかを書いてある）。

9.9.3 実効 SNR から STI へ

$$\text{SNR}_{\text{eff}} = 10\log_{10}\frac{m}{1-m} \quad ¤[-15,\ +15]\ \text{dB}¤¤¤¤$$

$$\mathrm{TI} = \frac{\mathrm{SNR}_{\mathrm{eff}} + 15}{30}, \qquad \mathrm{MTI}_k = \frac{1}{14}\sum_F \mathrm{TI}_{k,F}$$

$$\mathrm{STI} = \sum_{k=1}^{7} \alpha_k \,\mathrm{MTI}_k - \sum_{k=1}^{6} \beta_k \sqrt{\mathrm{MTI}_k \,\mathrm{MTI}_{k+1}}$$

α は各帯域の寄与、β は隣り合う帯域が持つ**冗長性**を差し引く分。 男声・女声で別の重みがあり（女声は 125 Hz の寄与がゼロ）、$\sum\alpha - \sum\beta = 1$。

6 バンド（125〜4k Hz）で計算したときは 8 kHz が無いので、 その帯域の α と関係する β を外して正規化し直し、警告を出す。

STI	評価（IEC 60268-16）
0.75 以上	優
0.60〜0.75	良
0.45〜0.60	可
0.30〜0.45	不可
0.30 未満	劣

検算：純粋な指数減衰・騒音なしなら $T = 1$ s で STI ≈ 0.59、 $T = 0.5$ s で 0.74、$T = 2$ s で 0.44 になる（文献値と一致）。

9.10 吸音材だけ変えた再計算 — どこから再開できるか（依頼 2026-08-21）

計算のうち**吸音率に依存するのはどこか**を整理すると、再開できる位置が決まる。

段階	吸音に依存するか	理由
① 音線追跡	**しない**	反射は鏡面反射（式は法線だけ）。受音しても打ち切らない
② 重複削除	しない	反射面の並びを比べるだけ
③ 経路の検証	**しない**	虚音源が成立するかは幾何（§7.2）
③' エネルギー	**する**	各反射で $\lvert R(\theta,\alpha)\rvert^2$ を掛ける（§7.3）
④ 以降	する	③' の結果を使う

したがって経路ごとに**反射面の並び $\{\text{¤}_k\}$ と各反射の入射角 $\cos\theta_k$**、 それに到来時刻・距離・到来方向を残しておけば、吸音材を変えた再計算は

$$E = \prod_k \lvert R(\cos\theta_k,\ \alpha_{\text{¤}_k})\rvert^2$$

の**掛け算だけ**で済む（`loop_noredundancy.energy_from_geometry`）。 掛ける順番が逆（経路の先頭から）になるが積なので値は同じで、 実測の差は丸めの 10^{-15} 程度。

★交差判定のパッチが材料で決まることに注意

交差判定は**同一平面パッチ単位**で、パッチは「同一平面＋辺で連結＋**同じ材料**＋ 同じ法線」でまとめている（材料で割らないと吸音率が引けない）。つまり

- 材料の**値**（吸音率）を変えるだけ → パッチは同じ → **経路も同じ**
- 材料の**分け方**が変わる → パッチの切れ目が動く → **経路も変わる**

そこで保存時に指紋（三角形の頂点と法線・パッチの分け方・音源・受音点・ 音線数・最大反射回数・受音球・両面判定）を残し、読むときに突き合わせる（`path_cache.fingerprint`）。**吸音率の値は指紋に入れない。**

9.11 定在波を反射回数で絞る・効いている壁を出す（依頼 2026-10-02）

9.11.1 反射回数で絞る（`--reflections`）

伝達関数 $H(f)$（式 2.67）は経路ごとの項の和なので、反射回数 k_n が 範囲 $[k_{\mathrm{lo}}, k_{\mathrm{hi}}]$ に入る経路の集合を K とすれば、 それだけを足したものも同じ形で書ける。

$$H_K(f) = \sum_{n \in K} a_n \, e^{-j2\pi f d_n / c}$$

$a_n = \sqrt{E_n/4\pi}/d_n$（空気吸収は $e^{-m(f)d_n/2}$ を掛ける）。 全反射を足すと、拡散したあとの音まで重なって山が埋もれるので、低次の反射だけにすると **どの平行面のモードか**が見えやすくなる。ただし

- 絞った H は**本来の応答ではない**（形状検討の目安）
- 周波数分解能は $\Delta f \approx c/\max d_n$ なので、**絞るほど粗くなる**

直方体の虚音源（`mode_shape --method box`）は虚音源ごとの反射回数 $|l-p|+|l|+|m-q|+|m|+|n-r|+|n|$ で同じように選ぶ。

9.11.2 その壁で反射が起きないとしたときの変化

ある壁（レイヤ・面）w で 1 回でも反射した経路の集合を S_w とし、

$$H_w(f) = \sum_{n \in S_w} a_n \, e^{-j2\pi f d_n/c}$$

とおく。壁 w の反射率を**どの入射角でも 0** にすると、S_w の経路はちょうど消え、 ほかの経路のエネルギーは変わらない（$E = \prod_k |R_k|^2$ の 1 因子が 0 になるだけ。§9.10）。 したがって

$$\Delta L_w = 20 \log_{10} \frac{|H(f) - H_w(f)|}{|H(f)|}$$

は近似ではなく、**その壁を $R = 0$（開口と同じ）にした計算そのもの**になる （`path_contribution.measures`。テスト [65] で両者が一致することを確かめている）。

- 負なら「その壁の反射を減らすと、その周波数の山が下がる」
- ★このプログラムの反射率は入射角で変わる（式 2.64）ので、垂直入射の $\alpha_n = 1$ でも

  斜めの反射は残る。ΔL_w は吸音材で得られる**効き目の上限**
- 1 本の経路は何枚も壁を通るので、壁ごとの ΔL_w を足しても全体にはならない
- 重なりのない分け方（通った面の組・到来方向・反射回数）では

  $\sum_g \mathrm{Re}(H_g \overline{H})/|H|^2 = 1$（「同位相の割合」）。負の組は打ち消す側にいる

$|\cdot|^2$ は `--width` の幅の中の数点で平均してから比を取る （山の位置が分解能の中で揺れるため）。

経路の対応づけ：`pulses.csv`（条件ごとのエネルギー）には通った壁が入っていないので、 `経路.npz`（壁の並び）と**反射回数・経路長・到来方向で並べて 1 対 1 に照合**する。 1 本でも合わなければ止める。npz を保存したときとモデルの形が違えば（指紋の `geometry`） 壁の番号が対応しないので、これも理由を告げて止める。

---

10. 数式とコードの対応表

数式	意味	書籍の式	Python	Fortran
$\mathbf{n} = \frac{\mathbf{v}_{12} \times \mathbf{v}_{13}}{\|\cdot\|}$	三角形の法線	図 2.12	`read_dxffile.read_model`	132〜283 行
交差数の偶奇（同一平面パッチ）	パッチ単位の内外判定	5.4.1 節	`mesh_method.PatchArrays`	—
`V=\frac13\oint \mathbf{r}\cdot\mathbf{n}\,dA`	容積（発散定理）	3.4 節	`read_dxffile.volume_from_normals`	—
`z_i = \frac{2i}{N}-1+\frac1N` ほか	等立体角の音線生成	図 2.11, A.15	`sound_ray.soundray_generator`	318〜326 行
$\mathbf{v} \cdot \mathbf{n} < 0$	壁に向かっているか	p.79	`loop_reflectionmesh.loop`	579 行
$d = -\mathbf{n} \cdot \mathbf{x}_1$	平面の方程式の定数	p.79	`mesh_method.parameter_d`	582 行
$t = -\frac{\mathbf{n} \cdot \mathbf{p}_0 + d}{\mathbf{n} \cdot \mathbf{v}}$	**交点パラメータ**	p.79	`mesh_method.parameter_t`	585 行
$\mathbf{q} = \mathbf{p}_0 + t\mathbf{v}$	交点座標	p.79	`mesh_method.node_renew`	593 行
`(\mathbf{u}\times\mathbf{e}_1)\cdot(\mathbf{u}\times\mathbf{e}_2)\le0`	三角形内部の判定	式 (2.50)	`mesh_method.collision_detection`	602〜625 行
$\min_j \|\mathbf{q}_j - \mathbf{p}_0\|$	最寄り面の決定	p.79	`loop_reflectionmesh.loop`	628〜636 行
`h\le r_c,\ 0\le s\le d_\mathrm{min}`	受音球の通過判定	p.84	`receiver_sphere.inside_sphere`	649〜663 行
$\mathbf{v}' = \mathbf{v} - 2(\mathbf{v} \cdot \mathbf{n})\mathbf{n}$	鏡面反射	図 2.10	`sound_ray.reflection_generator`	697〜700 行
$N = \sum m(m-1)^{i-1}$	虚音源の総数	式 (2.66)	（重複削除の根拠）	721〜841 行
$\mathbf{S}' = \mathbf{S} - 2(\mathbf{n} \cdot \mathbf{S} + d)\mathbf{n}$	虚音源の生成	図 2.18	`loop_noredundancy.loop`	900〜926 行
（経路の逆追跡）	虚音源の有効性判定	図 2.19	`loop_noredundancy.loop`	948 行〜
$R = \frac{(1+\sqrt{1-\alpha_n})\cos\theta - (1-\sqrt{1-\alpha_n})}{(1+\sqrt{1-\alpha_n})\cos\theta + (1-\sqrt{1-\alpha_n})}$	**斜入射の反射係数**	**式 (2.64)**	`sound_ray.energy_decay`	1093 行
$E_\mathrm{r} = \|R\|^2 E_\mathrm{i}$	反射後のエネルギー	式 (2.65)	同上	1093 行
$t = r/c$	到来時刻	p.86	`loop_noredundancy.backtrace_path`	1078〜1079 行
$E' = Ee^{-m_\mathrm{air}ct}$	空気吸収	—	`impulse.apply_air_absorption`	127 行
$H(f) = \sum \frac{\sqrt{E_n}}{ct_n} e^{-j2\pi f t_n}$	伝達関数	**式 (2.67)**	`impulse.transfer_function`（参照実装）	140 行
同上（時間領域に置いて FFT）	伝達関数（**既定・65 倍速い**）	8.2.1 節	`impulse.impulse_train`	—
$H \cdot H_t$	畳み込み（周波数領域）	—	`impulse.impulse_response`	190 行
$D(t) = \int_t^\infty h^2 d\tau$	**Schroeder 積分**	—	`reverberation.schroeder_integral`	ipls2rt 114〜

数式	意味	書籍の式	Python	Fortran
				117 行
$T_{30} = (t_{-35} - t_{-5}) \cdot 2$	残響時間	2.3.3	`reverberation.decay_curves`	ipls2rt 134 行
$T_{60} = \frac{0.161V}{-S\ln(1-\alpha)}$	Eyring 式（検算用）	2.3.3	（理論値。コードには無い）	—
$f = \frac{c}{2}\sqrt{(\frac{n_x}{L_x})^2 + (\frac{n_y}{L_y})^2 + (\frac{n_z}{L_z})^2}$	直方体の固有周波数	2.1	`plots.room_modes`	—
$f_s = 2000\sqrt{T/V}$	シュレーダー周波数	2.1	`plots.schroeder_frequency`	—
$H(f) = \sum a_n e^{-j2\pi f t_n}$	経路差からの積み上げ	9.7 節	`plots.pulse_spectrum`	—
$f = mc/\Delta$	経路差 Δ が強め合う周波数	9.7 節	`plots.mode_buildup`	—
$E_i = A_i e^{-md_i}/(4\pi d_i^2)$	受音点で受け取るエネルギー	9.8 節	`sound_level.received_energy`	—
$L_p = L_W + 10\log_{10}\sum E_i + 10\log_{10}\frac{\rho c}{400}$	**音圧レベル**	9.8 節	`sound_level.band_levels`	—
$L_p = L_W - 20\log_{10} d - 11$	自由音場（逆二乗）の物差し	9.8 節	`sound_level.freefield_level`	—
$\rho = pM/(RT)$	湿り空気の密度	9.8 節	`atmosphere.density`	—
$m(F) = \frac{\lvert\sum E_i e^{-j2\pi F t_i}\rvert}{\sum E_i}$	**変調伝達関数**	9.9 節	`sound_level.modulation_transfer`	—
$\mathrm{STI} = \sum \alpha_k \mathrm{MTI}_k - \sum \beta_k \sqrt{\mathrm{MTI}_k \mathrm{MTI}_{k+1}}$	**STI**	9.9 節	`sound_level.speech_transmission_index`	—
$E = \prod_k \lvert R(\cos\theta_k, \alpha_k)\rvert^2$	幾何からエネルギーを作り直す	9.10 節	`loop_noredundancy.energy_from_geometry`	—
$H_K(f) = \sum_{n \in K} a_n e^{-j2\pi f d_n/c}$	反射回数で絞った伝達関数	9.11 節	`frequency_response.select`	—
$\Delta L_w = 20\log_{10}\lvert H - H_w\rvert/\lvert H\rvert$	その壁で反射が起きないとしたときの変化	9.11 節	`path_contribution.measures`	—

11. 参考文献

★0. **Masahiro Toyoda and Yuta Sakayoshi, "Filter and correction for a hybrid sound field analysis of geometrical and wave-based acoustics", Acoust. Sci. & Tech. (2021)** — `参考文献/03_幾何音響・音線法/`。**PJ 担当者が著者**なので、 実装の根拠として最優先で参照する。

- **式(1)** … 斜入射のエネルギー反射率（7.3 節。 `sound_ray.energy_decay` ）
- **★式(2)** … 虚音源からの波の重ね合わせを**周波数領域で**行う

  $p = \sum \sqrt{E_n}\, e^{ikr_n}/r_n$（8.2 節。書籍 式(2.67) と同じもの）
- **第 3 節** … なぜ時間領域で足さないか。**サンプル時刻へ「寄せる」**と

音圧でなくエネルギーのインパルス応答になる（8.2.1 節でこの点を検証した）

- **第 5 節** … 帯域分割は**直線位相 FIR**（窓関数法）を使う。Butterworth /

  Chebyshev の IIR は位相が非線形でクロスオーバー付近が乱れる
  （`impulse.filter_bandpass`。なお `reverberation` 側の帯域分割は
  指標を測るためのもので別扱い）

1. IEC 60268-16 *Sound system equipment – Part 16: Objective rating of speech

   intelligibility by speech transmission index* — STI（9.9 節）の規格。
   変調周波数・帯域の重み・聴覚マスキング・評価区分はこれに従っている
2. 豊田政弘『建築音響物理学』関西大学出版部

   — 2.2 幾何音響理論（音線法・虚像法）、2.3 統計音響理論、3.1 吸音、補遺 A.15 立体角
   本 PJ の理論的な下敷き。抜粋 PDF は `参考文献/05_書籍/`
3. 「垂直入射吸音率と乱入射吸音率」 — `参考文献/01_吸音率・反射特性/`

   TODO D-7（残響室法吸音率への対応）の資料
4. 「軸対称等立体角26面体を用いた全方位の離散化」 — `参考文献/02_音線生成・方向離散化/`

   音線生成の代替手法
5. 元 Fortran コードが参照している文献（`backtrace.f90` 551 行のコメント）

   https://www.jstage.jst.go.jp/article/jsgs1967/36/1/36_1_3/_pdf

> PDF はいずれも Git 管理外です（public リポジトリのため）。 `参考文献/README.md` を参照してください。